

ANGULAR

Rev 3.3 del 03/03/2026

Introduzione ad Angular.js ed Angular 2	2
Angular 2: installazione creazione ed esecuzione di un'applicazione.....	3
Struttura dell'applicazione	4
La cartella src	5
App Component	6
Concetto di Decoratore	7
Il file tsconfig.json	7
Import di un file json	7
Utilizzo del debugger	7
Creazione automatica di nuovi componenti	8
Creazione manuale di un nuovo componente	9
La programmazione	10
Binding unidirezionale tramite le doppie graffe	10
Accesso alle risorse statiche	10
Le direttive Strutturali .ngIf e ngFor	11
Binding di un attributo HTML tramite []	13
Binding delle proprietà di stile tramite le direttive di attributo	14
Binding bidirezionale di un attributo HTML tramite ngModel	15
La gestione degli eventi	16
Utilizzo di bootstrap come libreria interna al progetto	16
Accesso agli elementi della pagina html	17
La comunicazione fra componenti	19
Property Binding tramite il decoratore @Input()	19
Creazione delle righe di una tabella	20
Event Binding tramite il decoratore @Output()	21
Ascolto di un evento da codice: il metodo subscribe()	22
Direttive personalizzate	23
ngForm e la validazione dell'input	26
I servizi	27
Creazione di un servizio base per l'invio delle richieste Ajax	27
Accesso ai dati ricevuti : l'oggetto Observable	29
Il controllo dell'applicazione tramite services	30
Il Routing	34
Routing delle pagine	34
Routes parametriche	37
Child routes	38
Lazy Loading Modules	41
Pubblicazione di un progetto Angular su firebase	42
Creazione di una apk angular - cordova	45

Angular

Angular 1 (noto oggi anche come Angular.js) è una semplice **libreria** javascript nata nell'ambito del progetto **mean** e mirata allo sviluppo del lato client. Fin da subito la principale prerogativa di Angular.js è stata quella di realizzare un **binding bidirezionale** tra una struttura dati in memoria e la rappresentazione della pagina html, evitando così tutte le operazioni di accesso agli elementi della pagina e conseguente caricamento dei dati.

Dalla versione 2 in avanti Angular è diventato un vero e proprio **framework** per lo sviluppo di applicazioni web lato client. La prima release è stata presentata nel **2016** grazie soprattutto all'impulso di Google. Angular 2 eredita alcuni aspetti base di angular.js ma è profondamente diverso e incompatibile.

Altre librerie simili ad Angular sono **vue** e **react**, entrambe tutt'ora distribuite sotto forma di libreria e dunque decisamente più leggere.

- **React** è quello più flessibile e, ad oggi, probabilmente il più usato.
- **Vue** ha una curva di apprendimento molto rapida ed è usato soprattutto nelle piccole realtà
- **Angular** ha una struttura molto solida e scalabile ed è quindi adatto a progetti più grandi

Principali caratteristiche di Angular - (rif attuale Angular 17)

- mentre Angular.js era basato su JavaScript, Angular utilizza il linguaggio **typescript** che è una astrazione di JavaScript molto più strutturata, cioè maggiormente orientata agli oggetti e che, tramite i tipi, consente un maggiore controllo sul codice. Il comando **ng build** "compila" l'applicazione trasformando i files typescript in files html / css / js tradizionali in modo che il **browser** li possa interpretare correttamente. La fase di compilazione è necessaria anche se si utilizza semplicemente JavaScript. La compilazione genera codice JavaScript standard in versione **ES5** e quindi garantisce piena compatibilità anche con i browser meno recenti
- Angular2 è completamente centrato sul concetto di **componente** e presenta una spiccata **modularità**, basata sulla possibilità di poter creare nuovi componenti o utilizzare facilmente componenti di terze parti. **Un componente è un bundle che contiene sia la grafica sia il corrispondente codice di elaborazione**, per cui la pagina html principale risulta estremamente concisa e si limita a richiamare i vari componenti scritti separatamente.
- Angular crea al suo interno una vera e propria **applicazione** che andrà a caricare via via le pagine html necessarie. Angular è intrinsecamente **single page**, però mediante un apposito meccanismo di routing interno, si presenta all'esterno come un **multipage** in cui ogni 'pagina' ha un suo path.
- Il 'motore' run time di Angular consente un **binding bidirezionale** real time tra il codice e gli elementi della pagina html, svincolando l'applicazione dalle solite problematiche tipiche della programmazione asincrona. C'è un demone che gira in background e si occupa di mantenere l'interfaccia grafica sempre perfettamente aggiornata rispetto al valore delle variabili interne.

Altre caratteristiche

- E' un progetto **open source** pubblicato su github a cui partecipano milioni di sviluppatori
- E' fortemente orientato al pattern **MVC**
- E' interamente scritto sulla base delle direttive di ECMAScript 2015 (**ES6**)
- E' di tipo **mobile first**, cioè garantisce elevate prestazioni per assicurare una interazione fluida sui dispositivi mobile. Supporta i principali eventi tipici del mobile (es touch e gesture)
- Per l'utilizzo di **Angular** esistono delle apposite GUI, ma il modo più diretto è la **linea di comando**

Guida base disponibile al seguente link: **Angular.io** / GetStarted / Setup

Add-on Esistono moltissimi componenti aggiuntivi (add-on) che consentono di estendere angular (griglie, calendari, etc). Ad esempio **Angular Material** è un toolkit che facilita la progettazione in Angular, fornendo una serie di componenti ottimizzati per Android già pronti per essere utilizzati. Per utilizzare Angular Material occorre semplicemente importare la libreria e poi utilizzare i vari componenti.

Prerequisiti: la presenza di node.js alla versione uguale o superiore a **22** (novembre 2025)

Installazione di Angular CLI

```
npm install -g ng // installa angular come semplice libreria
```

Viceversa

```
npm install -g @angular/cli
```

Installa una Command Line Interface che è un ambiente a riga di comando basato su nodejs per poter creare la struttura di un'applicazione Angular 2 configurata secondo le linee guida ufficiali. Anche **@angular/cli** è ospitato su github, per cui è possibile analizzare l'intero codice.

Visual Studio Code Extensions

Angular Snippets (John Papa)

Angular Language Service

I primi comandi

```
ng --help visualizza l'elenco dei comandi disponibili
ng --version restituisce il numero di versione (20.3 a novembre 2025)
ng version simile al precedente restituisce un set di informazioni più completo
ng new crea una nuova applicazione
ng serve fa una build e crea al volo un server http per testare la app
```

L'opzione **--help** dopo qualsiasi comando visualizza le opzioni disponibili per quel comando.

Esempio **ng generate component --help**

Creazione di una nuova applicazione

```
ng new ese01 --no-standalone
```

Questo comando installa tutte le librerie Angular necessarie, **tutte alla stessa versione della CLI** utilizzata per la creazione. Notare che un upgrade della CLI non provoca ovviamente un upgrade delle altre librerie, che devono pertanto essere reistallate. A quel punto conviene creare un nuovo progetto.

Attenzione che nel nome del progetto non sono ammessi spazi o underscore.

E' ammesso **solo il trattino**. Questo comando crea una applicazione pura angular (per web, non per smartphone). Alle varie domande poste il semplice INVIO significa accettare il default.

Le domande sono le seguenti:

- Abilitazione dello Strict mode (N)
- Abilitazione dell'Angular routing (N)
- Formato del file css (va bene normale **CSS** oppure **SCSS**)
- Do you want to enable Server-Side Rendering (SSR) and Static Site Generation **N**

All'invio viene creata la nuova applicazione, che consiste nella creazione di una **alberatura** principale compreso un file package.json sul quale, al termine, viene lanciato il comando **npm install** che provvederà ad installare tutte le librerie necessarie con le relative dipendenze. Inoltre viene creato ed inizializzato anche un repository locale di tipo git, compresi i file readme.md, .gitignore, etc)

Esecuzione dell'applicazione

Dalla cartella del progetto lanciare `ng serve --open`

Il comando `ng serve`, esegue un build temporaneo in memoria ed avvia l'esecuzione di un server locale nodejs sulla porta 4200 denominato "Angular live development server" che invia al browser le pagine compilate che verranno via via richieste. Inoltre avvia un monitoraggio costante su tutti i file che compongono il progetto. Nel momento in cui si modifica un qualsiasi file, **l'applicazione viene automaticamente ribuildata** e ricaricata dal browser senza necessità di eseguire un refresh e mantenendo lo stato corrente (**live-reload**). Notare che il refresh della pagina è di solito una operazione abbastanza onerosa perché comporta il reinserimento di tutti i dati necessari per arrivare alla pagina.

L'opzione `--open` apre automaticamente il browser alla url `http://localhost:4200/`

La pagina DEMO

La pagina demo presenta una serie di link molto interessanti che, fino ad Angular 16, erano i seguenti:

- Il primo link apre una applicazione di esempio legata ai supereroi
- Il secondo link apre un manuale della CLI
- Il terzo punta al blog ufficiale di angular
- I pulsanti inferiori visualizzano i principali comandi disponibili nella CLI

Inspector

Se con chrome si ispezionano i sorgenti dell'applicazione, si vede un index.html minimale, e poi c'è una serie di file js molto raffinati che contengono la traduzione del typescript programmato dallo sviluppatore. Nelle build di debug è anche possibile vedere i file typescript associati (src / app)

Nelle build di release questo non è possibile

Struttura di una applicazione Angular 17

La versione **Angular 17** (8 novembre 2023) presenta almeno 3 profonde innovazioni rispetto al passato:

- l'eliminazione di `app.module.ts` in cui veniva eseguito un `import` centralizzato di tutte le librerie necessarie. Ora viene invece utilizzata una modalità detta **stand alone** in cui ogni componente esegue autonomamente l'import di tutte le librerie che gli servono. In questo modo :
 - risulta più facile il riutilizzo del componente in altre applicazioni.
 - velocizza l'avvio che non deve più attendere il caricamento di tutti i **moduli 'generici'**.Notare che si tratta di una cosa ancora diversa rispetto al **Lazy Loading** che si vedrà più avanti. L'obiettivo è il medesimo, ma il Lazy Loading consiste nel non caricare tutti gli **user-components** all'avvio, ma caricarli soltanto nel momento in cui vengono utilizzati. Fino alla versione 19 quando si crea un nuovo progetto rimane ancora possibile crearlo "vecchia maniera", utilizzando l'opzione `--no-standalone` non più disponibile in Angular 20
- Le direttive `*ngIf` e `*ngFor` sono state sostituite con le direttive `@if` e `@for` (vedi più avanti).
- Ogni componente diventa depositario del proprio routing, con l'introduzione per ogni componente di due nuovi file, uno per la configurazione del routing e l'altro per l'impostazione delle routes.

Nella root di una applicazione Angular 17 sono contenuti i seguenti files / cartelle:

README.md contiene informazioni generiche su Angular ed un elenco dei passi che occorre fare per avviare l'applicazione, eseguire il build, etc.

.gitignore File da non sottoporre a versioning. Attenzione soprattutto a escludere `node_modules`

node_modules contiene tutti i moduli nodejs necessari per sviluppo ed esecuzione dell'applicazione

tsconfig.json File di configurazioni tipico di tutti i progetti nodejs

package.json contiene :

- il **nome** della app,
- il **numero di versione** impostato come 0.0.0 (release, major number (cambiano piccole cose), minor number (non cambia nulla ma c'è il fix di un problema))
- elenco degli scripts che si possono utilizzare (ng, ng new, ng serve, ng build, etc)
- elenco delle **dipendenze** La principale è sicuramente @angular/core
rxjs è il modulo che si occupa della gestione dei servizi rest (chiamate ajax)
- elenco delle **devDependencies**, cioè dipendenze che NON andranno a fare parte della build finale ma che sono necessarie in fase di sviluppo e di build. Ad esempio la CLI

angular.json contiene la configurazione base del progetto Angular e di come dovrà essere eseguito il build.

public E' la cartella in cui verranno salvati i file html creati dall'interprete

Tutti questi file in genere non richiedono modifiche.

La cartella src

E' la cartella che contiene la app vera e propria. Contiene sostanzialmente soltanto tre files :

index.html tipica pagina HTML in cui all'interno del <body> è contenuto un unico **selettore** denominato **<app-root>** definito all'interno del componente **app.component**
Rappresenta la single page di apertura dell'applicazione e deve essere lasciata così com'è.
Tutto il codice sarà scritto all'interno di **app.component**

style.css contiene gli stili globali dell'applicazione (in realtà gli stili vengono scritti principalmente all'interno dei componenti, per cui questo file conterrà solo gli stili generali comuni a tutti i componenti)

main.ts Molto alleggerito in Angular 17.

rappresenta il vero entry point dell'applicazione, simile al file **program.cs** delle applicazioni C#.

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

L'interprete Angular, in corrispondenza di ng serve, eseguirà questo file il quale in sostanza fa **due cose**:

- importa **AppComponent** e **appConfig**
- importa **bootstrapApplication** che serve per mandare in esecuzione **AppComponent**

In precedenza importava anche `app/app.module.ts` ed `environments/environment.ts`.

- Il file `environments / environment.ts` è costituito da una unica istruzione che setta a true/false una variabile denominata `environment.production` che indica se l'applicazione deve essere eseguita in modalità *produzione* oppure *debug*.
In caso di `environment.production=true` viene eseguita la `enableProdMode()` che disabilita tutte le modalità di debugging ed i console.log in modo da velocizzare l'esecuzione.
- Il file `app/app.module.ts` è un file di configurazione che definisce la classe **AppModule** che contiene l'elenco completo di tutti i componenti da caricare per l'esecuzione dell'applicazione

All'interno della sottocartella **app** viene definito il componente principale **AppComponent** con i suoi 5 files

La sottocartella app

App.component (dalla versione 20 semplicemente **App**) è il componente principale visualizzato da **main.ts**. Un **componente** è una classe (con iniziale maiuscola) con associato un **decoratore** **@Component** che 'trasforma' una classe in componente.

Associati ad **AppComponent** in **Angular 17** ci sono 5 files:

1) app.component.ts è il file più importante.

Definisce ed Esporta una classe **AppComponent** che realizza il componente:

```
import {Component} from '@angular/core';

@Component({
  selector: 'app-root',      // utilizzato in index.html
  standalone: true,          // dalla versione 20 non più configurabile
  imports: [RouterOutlet],
  templateUrl: './app.component.html', // HTML associato
  styleUrls: ['./app.component.css']    // CSS associato
})
export class AppComponent {
  title = 'ese01'; // property della classe
}
```

- Il decoratore **@Component** viene applicato alla classe **AppComponent** aggiungendo nuove caratteristiche alla classe stessa e trasformandola in un **Componente Angular**.
- Il decoratore **@Component** assegna le tre caratteristiche fondamentali di un componente Angular:
 - definisce un **selettore** che consentirà di utilizzare il componente all'interno di una qualsiasi pagina html.
 - assegna al componente un **file html** in cui verrà definita la struttura html del componente
 - assegna al componente un **file css** associato inizialmente vuoto.
- Invece di **templateUrl** e **styleUrls** si possono utilizzare **template** e **style** che, anziché collegare un file esterno, consentono di scrivere direttamente il codice html ed i relativi stili.
- La classe **AppComponent** viene 'esportata' tramite una istruzione **export** (**ES-modules**) e rappresenta l'oggetto che viene mandato in esecuzione da **main.ts**
- All'interno di **AppComponent** è possibile definire nuove Proprietà e Metodi personalizzati.

2) app.component.html definisce il contenuto html relativo al componente. All'interno del file html si possono inserire normali tag html oppure altri componenti angular creati appositamente. Al fine di migliorare la portabilità è raccomandato l'utilizzo di soli componenti angular.

3) app.component.css E' la pagina in cui si possono definire gli stili del componente. Attenzione che gli stili definiti in questo file valgono **SOLO** per i tag scritti staticamente nel file html e non per eventuali tag creati dinamicamente attraverso javascript. Gli stili da associare ad eventuali tag creati dinamicamente DEVONO essere scritti all'interno del file globale **styles.css**

4) app.config.ts File di configurazione del routing

5) app.routes.ts File di definizione delle routes

app.component.spec.ts Facoltativo, consente di definire dei test sull'applicazione

Concetto di Decoratore

Un decoratore, introdotto dal simbolo @ (at), è un particolare metodo di classe che **consente di aggiungere nuove caratteristiche all'oggetto a cui viene applicato**. Si aspetta come parametro sempre un json che conterrà informazioni diverse a seconda del tipo di decoratore.

Esistono diversi tipi di decoratore:

@Component() trasforma la classe in un Componente Angular utilizzabile nelle pagine html.

@ViewChild() consente alla classe di accedere agli elementi HTML della pagina

@Input() consente ad un Top Component di iniettare un valore ad un Bottom Component in fase di creazione

@Output() opposto al precedente. Consente ad un bottom componente di generare un evento che potrà essere intercettato dai top component con la conseguente possibilità di passare dei parametri dal bottom component al top component

@Injectable() rende la classe in cui è definito "iniettabile" al costruttore di un qualsiasi componente.

Il file tsconfig.json

Consente interessanti configurazioni relativamente al progetto corrente.

Per evitare controlli troppo stringenti si possono eseguire le seguenti impostazioni:

```
"strict": false,  
"strictPropertyInitialization": false,
```

La seconda, in caso di valore `true` (default), richiede l'inizializzazione esplicita di tutte le property.

Import di un file json

Qualsiasi componente può importare un file json mediante semplice **import** (esattamente come in node.js) :

```
import students from "../students.json" // posizionato nella cartella corrente
```

A tal fine occorre però impostare sempre all'interno del file **tsconfig.json** (sezione **CompilerOptions**) le seguenti configurazioni (esattamente come in node js):

```
"resolveJsonModule": true,  
"allowSyntheticDefaultImports" : true,
```

Utilizzo del debugger

E' possibile eseguire un **Debug** direttamente sul file typescript. Sulla finestra **Sources** aprire

```
webpack / . / src / app / student /student.ts
```

Se metto un breakpoint nel costruttore ci passerà n volte, una volta per ogni selettore `<student>` creato nella pagina html principale.

get started

<https://angular.dev/tutorials/learn-angular/1-components-in-angular>

Creazione di nuovi componenti

Invece di scrivere tutti il codice all'interno di AppComponent in genere si procede con la creazione di nuovi componenti che:

- Possono essere richiamati dentro AppComponent oppure dentro altri sottoComponenti
- Possono essere riutilizzati in altre applicazioni.

Ogni componente viene creato in una sua sottocartella specifica.

Ognuna di queste sottocartelle conterrà 3 files:

```
student.component.ts
student.component.html
student.component.css
```

Per creare un nuovo componente si può utilizzare la CLI di angular che fa tutto da sola.

Crea la sottocartella con il nome del componente con all'interno i 3 files precedenti.

A tal fine occorre **fermare il server** e digitare il seguente comando:

```
ng generate component student --skip-tests
ng g c student --skip-tests
```

Fino alla versione 18, in coda al nome del componente veniva automaticamente aggiunto il suffisso Component. Dalla versione 20 non è più così ma il suffisso può essere aggiunto a mano:

```
ng generate component studentComponent --skip-tests
```

L'opzione **--skip-tests** evita la creazione del file di test

Omettendo l'opzione **--skip-tests** il file di test può comunque essere cancellato in qualsiasi momento

Digitando **ng generate --help** viene visualizzato l'elenco dei possibili comandi

Contenuti del nuovo componente

All'interno del file html il wizard aggiunge semplicemente il seguente tag <p>

```
<p> student works! </p>
```

Il componente creato deve poi essere importato all'interno del vettore **imports** in tutti i componenti che lo usano (ad es AppComponent)

```
import { StudentComponent } from './student/student.component';
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, StudentComponent],
```

L'import in alto viene fatto in automatico.

Creazione manuale di un nuovo componente

All'interno della sottocartella `/app` creare manualmente una sottocartella `student` ed 5 files precedenti.

- `student.ts`, prendendo spunto dall'`app-component`, scrivere il seguente codice di dichiarazione ed esposizione del nuovo componente:

```
import { Component } from '@angular/core';
@Component ({
  selector: 'app-student',
  templateUrl: './student.component.html',
  styleUrls: ['./student.component.css']
})
export class StudentComponent {
  name = 'pippo';
}
```

Una classe, per poter essere visibile dagli altri moduli, deve SEMPRE essere esposta mediante

```
export class ClassName { }
```

- `student.html` definisce la struttura html del componente:

```
<p> Hi, my name is {{name}} </p>
```

I nomi sono arbitrari ma è consigliato mantenere la stessa simbologia usata dal wizard.

Al selettore si assegna sempre un nome del tipo `<app-componentName>`. Si aggiunge app davanti per non interferire con i selettori html (il tag `student` non esiste, ma altri nomi potrebbe già esistere fra i tag html, ad esempio `article`)

Nel momento in cui si digita `@` e si sceglie il decoratore dall'elenco a discesa, automaticamente viene aggiunto in alto l'import corrispondente.

Il parametro atteso dal decoratore è un json costituito dai tre campi indicati.

Come detto le chiavi `templateUrl` e `styleUrls` stanno ad indicare che i valori indicati di seguito contengono la URL di un file. La chiave `template` si aspetta invece come parametro l'intero contenuto del file html interamente racchiuso all'interno del **backtick** (apice singolo rovesciato ALT+96). Es:

```
template: `

```

Utilizzo del componente all'interno di un altro componente

```
<h2>Elenco Studenti</h2>
<app-student> </app-student>
<app-student> </app-student>
<app-student> </app-student>
```

Registrazione del componente all'interno del file `app.module.ts`

Ogni volta che si crea un nuovo componente occorre "registrarlo" all'interno del file `app.module.ts`

Le righe da aggiungere sono due:

```
import { StudentComponent } from './student/student.component';

declarations: [
  AppComponent,
  StudentComponent
],
```

Nel momento in cui si scrive `StudentComponent` visualStudioCode provvede automaticamente ad aggiungere l'import.

La programmazione

Utilizzo di proprietà e metodi all'interno del file html

Il file `.html` definisce la grafica della pagina.

- Può far uso di tag html oppure di componenti angular
- Può accedere a Property e Methods delle classe associata. L'accesso può essere eseguito indifferentemente CON o SENZA il `this` (normalmente senza)

Binding unidirezionale tramite le Doppie Graffe (interpolazione di stringa)

- Nell'html è possibile visualizzare il valore di una qualunque property ts tramite l'utilizzo di una coppia di **doppie graffe (binding unidirezionale)** all'interno delle quali si inserisce il nome della variabile associata: `{{nomeProperty}}`. Si parla di **interpolazione di stringa**.
`<span>{{title}} app is running!</span>`
- Le **Doppie Graffe** possono essere inserite in qualsiasi punto di una pagina html, sia all'interno di un testo sia all'interno di un qualsiasi tag o attributo.
- Le **Doppie Graffe** in ogni momento riflettono il valore della variabile associata, cioè **renderizzano** il valore della variabile all'interno della pagina html.
- Le **Doppie Graffe** oltre a proprietà e metodi del componente associato, al loro interno possono anche utilizzare **operatori ternari**, **operatori matematici** e **concatenamento di stringhe**

Gli eventi `ngOnInit()` e `ngOnDestroy()`

Ogni volta che un componente viene visualizzato sullo schermo, vengono richiamati prima il **costruttore** e, dopo, l'evento `ngOnInit()` che è un evento analogo a `form_load` di C# **richiamato al termine della creazione del componente ma prima del suo rendering grafico**.

La documentazione raccomanda di utilizzare il **costruttore** soltanto per inizializzare le proprietà della classe, ed utilizzare invece `ngOnInit()` per svolgere tutte le azioni di avvio, come ad esempio l'avvio di un servizio o la creazione di un subscriber di ascolto di un evento.

Nel momento in cui il componente viene sostituito a video da un altro componente, viene sempre anche rimosso dalla memoria, generando l'evento `ngOnDestroy()` che può essere utile per deallocare eventuali oggetti istanziati dentro `ngOnInit()`, come ad es i subscriber di ascolto di un evento.

Per poter utilizzare correttamente gli eventi `ngOnInit()` e `ngOnDestroy` occorre che la classe implementi le relative interfacce: `export class RecipeComponent implements OnInit{ }`

Accesso alle risorse statiche

Da **Angular 17** in avanti le risorse statiche, come ad esempio le immagini, devono essere salvate all'interno della cartella `public` che si trova allo stesso livello di `src` (una volta era `src/assets`). Il file di configurazione `angular.json` definisce dove andare a cercare le risorse statiche :

```
"assets": [{ "glob": "**/*",
              "input": "public" }],
```

Questa impostazione può essere modificata, così come si può inserire nel vettore un path aggiuntivo. Il path per accedere alle immagini da codice sarà semplicemente `"/nomeFile"`

```

```

Vale anche per l'icona della scheda

```
href="/vallauri.ico"
```

Le Direttive strutturali

Le direttive **strutturali** consentono di inserire delle istruzioni Angular all'interno dei tag HTML, istruzioni che permettono di modificare dinamicamente la **struttura** della pagina aggiungendo o togliendo elementi. Al loro interno possono utilizzare proprietà e metodi definite nella corrispondente classe type script.

La direttiva *ngIf

```
<p>
  Gestione Contatti
  <span *ngIf="nomeUtente"> Contatto: {{nomeUtente}} </span>
  // che significa
  <span *ngIf="nomeUtente != undefined"> Contatto: {{nomeUtente}} </span>
</p>
```

Come valore della direttiva *ngIf si può assegnare una variabile booleana definita all'interno della classe oppure una condizione. Se la variabile / condizione risulta vera il tag viene **visualizzato**, altrimenti **non** viene visualizzato.

La direttiva else

Se la condizione della **ngIf** risulta falsa, si può utilizzare all'interno della IF medesima una **else** tale da visualizzare un template alternativo nel modo seguente:

```
<p *ngIf="serverCreated; else noServer" >
  Server was created. Server name is: {{serverName}}
</p>
<ng-template #noServer>
  <p> Server is being created </p>
</ng-template>
```

#noServer crea un reference ad una sezione di codice html.

Se la variabile `serverCreated` assume il valore true, l'intero template viene rimosso dal DOM.

La direttiva @if

La nuova direttiva **@if** di Angular 17 è molto più semplice perché non è associata a nessun tag ma può essere scritta direttamente nell'html come fosse un tag.

Se la condizione è vera i tag interni alla IF vengono visualizzati, altrimenti no.

Parallelamente alla direttiva **@if** è stata introdotta anche una direttiva **@else**

```
@if (isLoggedIn) {
  <app-student> </app-student>
}
@else {
  <p>Please log in</p>
}
```

La direttiva *ngFor

Supponendo di definire all'interno del file ts un vettore enumerativo contatti, si possono visualizzare tutti i contatti all'interno di un contenitore nel modo seguente:

```
<div >
  <p *ngFor="let contatto of contatti; let i = index" >
    <span>{{contatto.nome}} {{contatto.cognome}}, {{contatto.eta}} anni</span>
  </p>
</div>
```

Il ciclo FOR definisce un **cursore** contatto che scorre tutti i contatti presenti nel vettore enumerativo. Ad ogni ciclo **visualizza il tag <p> in cui si trova** , replicando anche tutti i tag interni.

All'interno del ciclo FOR, in coda alle istruzioni canoniche, è possibile definire e valorizzare nuove variabili. All'interno del ciclo FOR la parola chiave **index** rappresenta l'indice del record corrente all'interno della scansione (a base 0).

Il ciclo **ngFor** consente esclusivamente **l'iterazione attraverso una collezione o un vettore** .

Non è consentito un ciclo diretto del tipo `*ngFor="let i=1; i<20; i++"` che può essere simulato nel modo seguente (orribile): `<p *ngFor="let number of [1, 2, 3, 4, etc.]"`

La direttiva @for

Siccome il ciclo *ngFor presentava alcuni problemi legati alle prestazioni (dalla presentazione ufficiale di @for: *we often see performance problems in apps*) in **angular 17** è stato introdotto il ciclo **@for** che :

- Introduce un campo **track** obbligatorio a cui occorre passare un identificativo che identifichi univocamente i vari record generati dalla scansione, Questo track serve ad Angular per migliorare le prestazioni del real time rendering. Può essere l'ID del record, oppure il record medesimo, oppure in caso estremo, qualora non esistano altre soluzioni, il **\$index** della scansione.
- Non è più applicato su un componente ma può essere applicato liberamente all'interno della pagina html e, ad ogni scansione, replica l'intero contenuto html. Diventa pertanto molto simile al ciclo **for of** di javascript

```
<ul>
  @for (color of colors; track color) {
    <li>{{ color }}</li>
  }
</ul>

<ul>
  @for (course of courses; track course.id) {
    <li>{{ course }}</li>
  }
</ul>

// Nel caso item non sia univoco
<ul>
  @for (item of vect; track $index) {
    <li>{{ item }}</li>
  }
</ul>
```

Attenzione che nemmeno questo nuovo ciclo **@for** risolve il problema del **table-row** di pagina 19.

Binding di un attributo HTML tramite []

Oltre all'interpolazione di stringa, per collegare un **attributo html** al valore di una variabile typescript si può utilizzare il cosiddetto Property Binding che consiste nello scrivere il nome dell'attributo HTML tra **parentesi quadre** e poi assegnare come valore il nome della variabile typescript.

Le **parentesi quadre** stanno ad indicare che nella stringa di destra non è contenuto un valore diretto, ma una **variabile** o comunque una sezione typescript da interpretare.

Le seguenti due sintassi sono praticamente equivalenti:

```

<img [src]="recipe.imagePath">
<input type="text" [value]="recipeName">
```

- Il **Property Binding** rispetto alle `{{ }}` è più generale (può essere applicato anche alle classi) e gestisce meglio le eccezioni (es `img` non trovata) per cui, a parità di condizioni, risulta preferibile.
- Il **Property Binding** può essere **SOLO unidirezionale** dal codice verso l'interfaccia. **NON può diventare bidirezionale** aggiungendo delle semplici parentesi tonde all'interno delle parentesi quadre. Le parentesi tonde possono anche essere inserite, ma il flusso rimane unidirezionale.

Attributi Booleani

Per gli attributi booleani a destra dell'assegnazione si può impostare una condizione. Se la condizione risulta `true` l'attributo viene applicato, altrimenti **NON** viene applicato:

```
<button [disabled]="studentname.length==0" (click)="esegui()">
<button [disabled]="studentName===' ' " (click)="esegui()">
```

Classi

Il Property Binding, come accennato, può essere applicato anche alle classi:

```
<div [class.collapse]="isDisabled">
```

Cioè la classe 'collapse' viene applicata se la variabile booleana `isDisabled` è `true` e viceversa. *Alternativo a `ngClass`.*

Utilizzo di una funzione

Sia nel caso del Property Binding tramite `[ ]` sia nel caso dell'interpolazione di stringa `{{ }}`, come valore, anziché una singola variabile, è possibile richiamare una funzione scritta nel file `.ts`. Si consideri il seguente esempio in cui il valore da applicare a `.src` varia a seconda che si abbia una immagine binaria o base64:

```
<img [src]='checkBase64()'>
checkBase64() {
  if (!imgPath.startsWith('data:image/')) // cioè non è base64
    return `assets/img/${imgPath}`
  else return imgPath
}
```

[ngValue]

In molti casi, come ad esempio nel caso dei **value** delle singole option di un tag select, non è consentito memorizzare un json all'interno della proprietà value (che nel caso produrrebbe errore).

Questa limitazione può essere superata utilizzando non `[value]` ma `[ngvalue]` il quale accetta anche oggetti. Nel caso del tag select il json della voce selezionata viene rimandato alla proprietà **ngValue** del tag select che diventa anche la proprietà predefinita linkata a `ngModel`.

Binding delle proprietà di stile tramite le Direttive di Attributo

Il valore delle proprietà di stile può essere gestito tramite le seguenti **Direttive di attributo**:

[ngStyle] (Direttiva di Stile)

[ngClass] (Direttiva di Classe)

Queste direttive devono sempre essere scritte fra parentesi quadre perchè all'interno della **stringa** di destra si aspettano una sezione typescript contenente, in entrambi i casi, un **json**.

In **Angular 17**, all'interno del vettore degli import del componente, occorre aggiungere manualmente le classi **NgStyle** e **NgClass** oppure **CommonModule**.

[ngStyle]

Utilizzato all'interno di un tag html, consente di definire le proprietà di stile in modo parametrizzato. All'interno di un tag ci può essere un unico **[ngStyle]** che si aspetta come parametro un **json** contenente alcune proprietà di stile da valorizzare tramite il contenuto di altrettante variabili.

```
<p [ngStyle]="{'backgroundColor': student.gender=='F' ? 'pink' : 'cyan',  
              'fontWeight' : nVulnerabilities > 10 ? 'bold' : 'normal'}" >
```

Alla proprietà backgroundColor viene assegnato un valore differente a seconda che la successiva espressione booleana risulti vera oppure falsa. Al posto di 'backgroundColor' 'pink' 'cyan' si possono utilizzare delle variabili definite nel file typescript.

[ngClass]

Consente di assegnare una più classi al tag HTML nel quale viene inserita. All'interno di un tag ci può essere un unico **[ngClass]** che si aspetta come parametro un **json** contenente questa volta dei nomi di classi in cui ad ogni classe viene assegnata una espressione booleana. Se l'espressione booleana risulta vera la classe viene applicata, altrimenti no.

```
<p [ngClass]="{'female': student.gender=='F',  
              'male': student.gender=='M',  
              'underlined': student.city=='Fossano'}" >  
  
<p [ngClass]="{'green':serverStatus=='online',  
              'red':serverStatus=='offline',  
              'blinking':nErrors >10 }" >
```

Per applicare semplicemente una classe ad un tag è possibile anche utilizzare il PropertyBinding come indicato alla pagina precedente: `<div [class.collapse]="isDisabled">`

Spostamento del codice nel file TS

Invece di inserire il codice all'interno della pagina HTML, una soluzione più strutturata consiste nel creare all'interno del file TS un metodo che restituisca appositi valori da utilizzare all'interno di ngStyle / ngClass

```
<p [ngStyle]="{'backgroundColor':getColor(), 'color':'red'}" > </p>  
getColor(){ return this.student.gender=='F' ? 'pink':'cyan' }
```

oppure si può definire una funzione che restituisca un intero json con tutte le proprietà di stile:

```
<p [ngStyle]="getStyle()" > </p>  
getStyle(){  
    const backColor:string = this.student.gender=='F' ? 'pink':'cyan';  
    const color:string = this.student.gender=='F' ? 'black':'white';  
    return {backgroundColor : backColor, color : color }; }
```

Binding bidirezionale di un attributo HTML tramite [(ngModel)]

La direttiva `ngModel` consente di eseguire un **binding bidirezionale** tra la proprietà `value` di un controllo della pagina html ed una variabile typescript.

Vale SOLO per i controlli, cioè per i tag di input che dispongono dell'attributo `value`.

Gli altri tag infatti non fanno input ma solo visualizzazione, per cui è sufficiente l'interpolazione.

```
<input type="text" [(ngModel)]="userName">
```

Omettendo le tonde intorno ad `ngModel`, il binding diventa **unidirezionale**, dal codice verso l'interfaccia e diventa equivalente al Property Binding eseguito tramite parentesi quadre `[value]`

Nota: La seguente sintassi NON è ammessa: `<input type="text" [(value)]="userName">`

ngModel nel caso di radioButton, checkBox e listBox

```
<input type="radio" value="M" [(ngModel)]="userGender">  
<input type="radio" value="F" [(ngModel)]="userGender">
```

Se la variabile `userGender` assume lo stesso valore del `value` del radio Button, allora il radio verrà impostato a `checked` e viceversa. Allo stesso modo se dall'interfaccia grafica si seleziona un certo radioButton, automaticamente la variabile `userGender` assumerà lo stesso valore del `value` del radioButton selezionato.

Per quanto concerne i **checkbox**, indipendentemente dal `value`, la variabile potrà assumere soltanto i valori `true/false`. Se la variabile è `true` il checkbox sarà selezionato e viceversa

Per quanto concerne i **listBox**, il funzionamento è lo stesso dei radioButtons. Il listBox dispone della comoda proprietà riassuntiva `value` che viene 'collegata' alla variabile. Nel caso in cui la variabile contenga un value valido, il listBox automaticamente selezionerà quella opzione. Se le opzioni non hanno un value, il value del listBox coinciderà con l'`innerText` dell'opzione selezionata.

Registrazione di FormsModule all'interno di app.module.ts

Per poter utilizzare `[(ngModel)]` è necessario importare manualmente il componente **FormsModule** all'interno del file `app.module.ts`

```
@NgModule({  
  declarations: [  
    AppComponent,  
  ],  
  imports: [  
    BrowserModule,  
    FormsModule  
  ],  
})
```

Dalla versione 17 in avanti occorre invece importare **FormsModule** direttamente all'interno del componente che intende farne uso, sia negli import iniziali, sia negli imports di `@Component`

```
import { FormsModule } from '@angular/forms';
```

```
@Component({  
  selector: 'app-root',  
  standalone: true,  
  imports: [RouterOutlet, FormsModule],  
})
```


La gestione degli Eventi

Per quanto riguarda gli eventi, è sufficiente scrivere il nome dell'evento all'interno di parentesi tonde:

```
<button (click)="assegnaNome()"> cliccami </button>
```

In corrispondenza di ogni click verrà richiamata la procedura "assegnaNome()" scritta nel file .ts. Notare che la procedura deve essere scritta come stringa con le tonde finali, esattamente come si fa per richiamare una procedura javascript dall'interno di una pagina html: onClick = "assegnaNome()" Si tratta infatti di un semplice richiamo diretto a procedura.

Passaggio di un parametro alla funzione di evento

Trattandosi di un richiamo diretto e non di un puntatore, è possibile passare alla funzione di evento qualsiasi parametro visibile al momento del richiamo:

```
<a *ngFor="let user of users" (click)="onClick(user)"> . . . </a>
```

Utilizzo di bootstrap come libreria interna al progetto

Per poter utilizzare bootstrap all'interno del progetto sarebbe sufficiente specificare il link CDN all'inizio di tutti i file.html oppure scaricare bootstrap in locale ed inserire il link in tutte le pagine. In alternativa, è possibile linkare la libreria direttamente al progetto rendendola così fruibile in tutte le pagine.

Lanciando dalla cartella principale di lavoro il comando,

```
npm install bootstrap@3 // 3.4.1
```

bootstrap viene installato nella sottocartella node_modules con relativo aggiornamento del package.json

A questo punto occorre però referenziare bootstrap all'interno dell'applicazione, andando a 'registrare' bootstrap all'interno del file **angular.json** che contiene tutte le configurazioni relative al progetto.

All'interno del file angular.json cercare **styles**

In questa sezione sono elencati tutte le librerie css che devono essere utilizzate dall'applicazione:

```
"styles": [      // riga 26
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "src/styles.css"
],
```

Aggiungere il riferimento a bootstrap prima del riferimento al file src/styles.css, in modo che gli stili personali siano prevalenti rispetto a quelli definiti all'interno di bootstrap.

La stessa cosa si potrebbe fare alla riga 86 relativa alla sezione di test, ma non è strettamente necessario.

Se si usasse anche la componente js di bootstrap, occorrerebbe fare la stessa cosa anche per il js:

```
"scripts": [
  "node_modules/bootstrap/dist/js/bootstrap.js"
],
```

Ogni volta che si fanno delle modifiche al file **angular.json** occorre **riavviare** il server.

Per vedere l'effettivo funzionamento di bootstrap si può inserire un button con una classe bootstrap:

```
<button class="btn btn-primary"> invia </button>
```

Accesso agli elementi della pagina html

A differenza di jQuery, la modalità di lavoro di Angular è tale per cui non si ha quasi mai la necessità di accedere ad un elemento della pagina HTML.

Rimane però il fatto che in alcune situazioni particolari potrebbe servire.

Accesso da codice al puntatore dell'oggetto che ha scatenato l'evento

Il **this** non è utilizzabile in quanto gli eventi non sono associati tramite **addEventListener()**, ma sono semplicemente 'richiamati' dall'html. Per accedere dalla procedura di evento al puntatore dell'elemento che ha scatenato l'evento ci sono due possibilità:

1) utilizzo di event.target

A tutti gli eventi javascript viene automaticamente iniettato un parametro **event** che al suo interno contiene una proprietà **event.target** che rappresenta il puntatore all'elemento che ha scatenato l'evento.

Il parametro **event** non ha altre proprietà particolarmente interessanti per cui, se si ha a disposizione il **this**, il parametro **event** perde di significato.

In Angular il passaggio del parametro **event** alla procedura di evento non è automatico ma, in fase di chiamata, occorre utilizzare la parola chiave **\$event** che indica ad Angular di propagare i parametri alla procedura di evento. In questo modo la procedura di evento riceverà il parametro **event** che potrà rinominare a suo piacimento (ed esempio **e**) ed utilizzare per accedere al parametro **e.target**

Esempio di click su un listbox:

```
<select (change)="visualizzaRecord($event)">
visualizzaRecord(e) {
    console.log(e.target.value) // value della voce selezionata
}
```

Insieme a **event** è possibile passare anche altri parametri aggiuntivi:

```
<select (change)="visualizzaRecord($event, contest)">
```

2) utilizzo di un segnaposto (Template Reference)

In fase di creazione di un tag è possibile salvarsi un riferimento tramite l'utilizzo di un segnaposto (**Template Reference**), riferimento che potrà poi essere passato alla procedura di evento.

Riscrittura dell'esempio precedente

```
<select #1stCitta (change)="visualizzaRecords(1stCitta)">
visualizzaRecords (ref) {
    console.log(ref.value)
}
```

Accesso agli altri elementi della pagina

Per accedere in modo diretto agli elementi della pagina HTML esistono due appositi decoratori che si aspettano come parametro un segnaposto (**Template Reference**) tipo quello visto poc'anzi.

- `@ViewChild` restituisce il primo matching element all'interno della pagina
- `@ViewChildren` restituisce tutti gli elementi individuati come una `QueryList of items`

Esempio

Per accedere ad un textbox avente come segnaposto

```
<input type="text" #txtName >
```

occorre definire nel file di classe una variabile di tipo `ElementRef` (puntatore a oggetto che andrà a 'wrappare' l'oggetto html) preceduta dal decoratore `@ViewChild`

```
import { ViewChild, ElementRef } from '@angular/core';  
@ViewChild("txtName") _txtName! : ElementRef<HTMLElement>;
```

cioè definiamo una variabile denominata `_txtName` di tipo `ElementRef` che punta all'elemento HTML avente come segnaposto `"txtName"`.

A questo punto in qualunque evento si può utilizzare la variabile `_txtName` seguita dalla property `nativeElement` che è l'equivalente del metodo `jQuery .get()`, cioè restituisce un puntatore javascript 'pulito' all'elemento corrente.

```
this._txtName.nativeElement.disabled=true;
```

Il punto esclamativo

Il punto esclamativo in coda alla variabile `_txtName` "toglie" alla variabile la possibilità di assumere il valore null perché sappiamo con certezza che verrà inizializzata prima di essere utilizzata cioè, in questo caso, che nella pagina HTML **esiste** un elemento avente come **ID** `txtName`. Se per caso fosse **undefined**, proprietà e metodi successivi semplicemente non verranno eseguiti.

Accesso ad un bottom component

Sempre tramite `ViewChild` è anche possibile accedere ad un bottom component utilizzato all'interno della pagina. Nell'esempio si accede al primo `StudentComponent` definito nella pagina html:

```
@ViewChild(StudentComponent, {static:false}) _child : StudentComponent;
```

L'opzione `{ static:false }` fa sì che la variabile `_child` sia aggiornata in corrispondenza di ogni variazione sul componente figlio

In alternativa è sempre possibile utilizzare la sintassi precedente assegnando all'istanza del `ChildComponent` un semplice segnaposto (**Template Reference**):

```
<child-component #child></child-component>  
@ViewChild("child", { static: true }) _child: ChildComponent;
```

La comunicazione fra componenti

Property Binding tramite il decoratore @Input()

Un **Top Component** (che sta sopra), nel momento in cui utilizza all'interno del proprio file html il selettore di un **Bottom Component** (che quindi vivrà all'interno del Top Component), può passare al Bottom Component uno o più parametri definiti al momento stesso dell'utilizzo del selettore.

A tal fine il Top Component deve definire tra parentesi quadre una nuova variabile (in questo caso denominata genericamente [**nuovoStudente**]) alla quale, mediante un **Property Binding** andrà ad assegnare il contenuto di una variabile di ciclo oppure di una variabile definita nella classe.

students

```
<app-student *ngFor="let student of students" [ nuovoStudente ]="student.name">
</app-student>
```

student

Il Bottom Component dovrà dichiarare la Property **nuovoStudente** tramite il decoratore @Input () che lo avvisa che questa variabile verrà passata dalla classe dal Top Component in fase di dichiarazione del componente. La variabile **nuovoStudente** potrà poi essere normalmente utilizzata all'interno della classe e della pagina html associata

```
@Input () nuovoStudente:string
```

Si parla di **Property Binding** in quanto la comunicazione avviene mediante il passaggio di una Property dal Top Component al Bottom Component. Tale property viene detta anche target property.

ngOnInit ()

Attenzione che le variabili ricevute tramite @Input () **NON** sono visibili all'interno del costruttore, ma sono invece visibili all'interno di **ngOnInit ()** che viene richiamato DOPO rispetto alla valorizzazione dei parametri di tipo @Input().

Esempi

In quest'esempio il Top Component è una **lista di ricette** che, mediante un ciclo ngFor, genera tanti recipe-item passando come parametro l'intera ricetta che sarà visualizzata all'interno del recipe-item:

recipe-list

```
<app-recipe-item *ngFor="let item of recipes" [ recipe ]="item">
</app-recipe-item>
```

recipe-item

```
@Input () recipe:RecipeModel
```

Sequenza completa di caricamento di un componente

1. Istanza della classe (**recipe-list**) con richiamo del costruttore
2. Valorizzazione delle eventuali variabili in input (dichiarate tramite il decoratore @Input ())
3. Richiamo dell'evento **ngOnInit ()**
4. Rendering grafico del componente con esecuzione del ciclo FOR di istanza degli eventuali molteplici sottocomponenti (**recipe-item**) *per ognuno dei quali* verrà automaticamente avviata la presente sequenza di caricamento. All'interno del componente principale, in corrispondenza di ogni sottocomponente, viene anche aggiunto nella classe l'eventuale subscriber di ascolto di un evento

Notare la differenza tra le seguenti sezioni:

Sezione 1

```
<app-book *ngFor="let book of books" [book]="book"> </app-book>
```

Per ogni **book** presente nel vettore crea un componente **<app-book>** e gli inietta **book**

La struttura di **<app-book>** sarà la seguente:

```
<div class="riga">
  <span>{{book.author}} </span>
  <span>{{book.title}} </span>
</div>
```

Sezione 2 - Creazione delle righe di una tabella

```
<tr *ngFor="let book of books" >
  <app-book [book]="book" > </app-book>
</tr>
```

In cui **<app-book>** dovrebbe creare i TD.

Però non funziona perché **<app-book>** crea uno strato intermedio che maschera i TD

Per risolvere il problema si può attuare la seguente soluzione

```
<tr *ngFor="let book of books" ref [book]="book"> </tr>
```

che sostanzialmente definisce una variabile **ref** che punta al tag TR corrente

Nel file **app-book.ts** occorre eseguire la seguente impostazione:

```
@Component({
  selector: '[ref]',
  templateUrl: './book2.component.html',
  styleUrls: ['./book2.component.css']
})
```

Cioè, anziché assegnare un nuovo nome al selettore, si fa in modo che il selettore vada a puntare ad un tag esistente, cioè al tag attualmente puntato dalla variabile **ref**

In pratica il nostro componente 'estende' le funzionalità del tag **<tr>** : ciò che verrà scritto dentro **<app-book>**, cioè i vari tag **<td>**, verranno inseriti direttamente all'interno del tag **<tr>**

Event Binding tramite il decoratore @Output()

L'operazione inversa che consente di passare un parametro da un Bottom Component verso un Top Component è decisamente più complessa.

Si parla questa volta di **Event Binding** in quanto la comunicazione avviene attraverso la generazione di un evento custom emesso dal Bottom Component verso il Top Component.

Il Bottom Component (nell'esempio la barra di navigazione <app-header>) deve generare un evento custom al quale passa come parametro i valori da inviare al top component (nell'esempio app-component) . Codice del bottom component:

```
<a href="#" (click)="onSelectClick('recipes')">Recipes</a>
<a href="#" (click)="onSelectClick('shopping-list')">Shopping List</a>

// dichiarazione di un evento di tipo EventEmitter a cui passeremo una string
@Output() featureSelected = new EventEmitter<string>();

// in corrispondenza del click generiamo l'evento e gli passiamo la stringa
onSelectClick(feature: string) {
  // generazione dell'evento
  this.featureSelected.emit(feature)
}
```

EventEmitter è un **Typed Object**, cioè occorre specificare quale o quali tipi di parametri passeremo a questo evento. Nell'esempio si passa una semplice stringa identificativa, ma si potrebbero passare anche **più parametri** o eventualmente **nessuno** (<void>)

L'oggetto **featureSelected** è preceduto dal decoratore **@Output** che fa sì che questo evento possa essere propagato al top component, cioè a tutti i Top Component che stanno utilizzando il componente corrente.

Il metodo **emit()** dell'oggetto **featureSelected** è quello che genera fisicamente l'evento passandogli come parametro il valore feature ricevuto da onSelect()

Attenzione a scegliere EventEmitter da Angular Core, perché ci sono altri oggetti diversi con lo stesso nome

Il top component (nell'esempio app-component), all'interno dell'html, al momento della dichiarazione del bottom component (<app-header>) deve mettersi in ascolto dell'evento custom generato dal bottom component e poi gestirlo:

```
<app-header (featureSelected)="onNavigate($event)"> </app-header>

loadedFeature:string='recipes'
onNavigate(feature:string){
  this.loadedFeature = feature
}
```

In corrispondenza dell'evento **featureSelected** viene richiamato il metodo onNavigate() definito all'interno del file di classe

Al metodo onNavigate() viene passato come parametro **\$event** che è quello visto nella sezione di gestione degli eventi, cioè una parola chiave che indica ad angular di **propagare i parametri alla procedura di gestione dell'evento** (che per default non vengono propagati).

loadedFeature è una stringa che alla fine conterrà il testo del pulsante cliccato all'interno del bottom component ("recipes" oppure "shopping-list")

Passaggio di un json

Al metodo `emit()` in realtà è possibile passare anche un intero json per cui, se serve passare più parametri, si può passare un intero json creando più campi per ogni informazione da passare:

```
@Output() featureSelected = new EventEmitter<any>();

onSelect(feature: string) {
  let json = {"feature":feature, "cod":97}
  this.featureSelected.emit(json)
}
```

Note

Notare che l'evento viene propagato al componente superiore (top component) ma non ad eventuali altri componenti ancora superiori. Se il parametro deve essere propagato anche a componenti successivi, il componente intermedio deve a sua volta generare un evento da inviare al componente extra-superiore e così via.

Questa tecnica degli eventi custom va bene per passare parametri fra componenti direttamente interconnessi. In caso di variabili che devono essere condivise fra molti componenti conviene utilizzare i **services** che consentono di condividere i dati fra più componenti.

Ascolto di un evento da codice: il metodo subscribe()

Quando un Bottom Component genera un evento tramite il metodo `emit()`, il Top Component può mettersi all'ascolto all'interno dell'html come indicato negli esempi precedenti, ma può mettersi in ascolto anche da Type Script.

A tal fine deve disporre di un puntatore al Bottom Component ottenibile tramite il decoratore `ViewChild`:

```
@ViewChild(StudentComponent, {static:false}) _child : StudentComponent;
```

L'opzione `{ static:false }` fa sì che la variabile `_child` sia aggiornata in corrispondenza di ogni variazione sul componente figlio

A questo punto il Top Component può mettersi in ascolto dell'evento nel modo seguente:

```
ngOnInit(): void {
  _child.featureSelected.subscribe(
    (feature:string) => {
      this.loadedFeature = feature
    }
  )
}
```

In pratica il componente corrente (nell'esempio `app-component`), si mette in ascolto permanente dell'evento `featureSelected` generato dal componente puntato da `_child`.

Tutte le volte che si verifica l'evento `featureSelected` verrà eseguito il metodo `subscribe` di gestione dell'evento al quale viene iniettato il parametro `feature:string` che, nell'esempio, viene semplicemente salvato all'interno di una variabile locale.

Per ottenere il puntatore al Bottom Component si potrebbe anche far uso dei Template Reference :

```
<component1 #disabler></component1>
<component2 #disabler></component2>
<component3 #disabler></component3>

@ViewChildren('disabler') mycomponents: QueryList<any>;
this.mycomponents.forEach((element)=>{element.disable();})
```


Direttive di attributo personalizzate

Oltre a nuovi Componenti è possibile anche creare nuove **Direttive personalizzate** che consentono sostanzialmente di creare nuovi attributi HTML (oppure nuove classi) associati a specifiche property CSS

Una direttiva personalizzata può essere creata manualmente come semplice classe oppure può essere autogenerata tramite CLI fermando il server ed utilizzando il seguente comando:

```
ng generate --help
ng generate directive shared/highlight --skip-tests
```

- Viene generata una cartella **shared** (facoltativa) con dentro il file **highlight.directive.ts**
- La Direttiva si chiamerà semplicemente con il nome del file: **Highlight**

Alla classe **Highlight** viene automaticamente applicato il decoratore `@Directive()` il quale associa alla Direttiva un selettore per l'utilizzo della Direttiva stessa all'interno di un qualunque componente ospite

```
@Directive({
  selector: '[appHighlight]'
})
```

La Direttiva veniva anche automaticamente registrata fra le `declarations: []` del file `app.module.ts`

La sintassi di utilizzo è sostanzialmente la stessa, indipendentemente dal fatto che si tratti di una Direttiva di Attributo oppure di una Direttiva di Classe.

Direttiva di Attributo Highlight

Definiamo una direttiva **Highlight** che gestisca il colore di sfondo del componente ospite e lo modifichi in corrispondenza dell'hover.

```
export class Highlight {
  @Input() defaultColor?: string = 'LightCyan'
  @Input() highlightColor?: string = 'Cyan'

  @HostBinding('style.backgroundColor') backgroundColor=:string;

  ngOnInit() {
    this.backgroundColor = this.defaultColor;
  }
  @HostListener('mouseenter') evidenzia(event: Event) {
    this.backgroundColor = this.highlightColor;
  }
  @HostListener('mouseleave') rilascia(event: Event) {
    this.backgroundColor = this.defaultColor;
  }
}
```

In pratica il selettore **appHighlight** può essere usato come un attributo html al quale è possibile passare due parametri :

- il parametro **defaultColor** che definisce il colore di sfondo da assegnare al componente ospite in condizioni di riposo (cioè al di fuori del mouseover).
- il parametro **highlightColor** che definisce il colore di sfondo da assegnare al componente ospite in corrispondenza del mouseover.

Il decoratore `@HostBinding()` fa sì che la **CSS Property backgroundColor** del componente ospite venga 'collegata' al campo `.backgroundColor` della classe corrente.

Questo campo viene inizializzato al valore assegnato al campo **defaultColor** e, in corrispondenza del **mouseover**, assumerà il valore del campo **highlightColor**

Notare che **@HostListener** è un decoratore di metodo. In TypeScript un decoratore di metodo può essere applicato **solo** a un metodo della classe, non a una funzione anonima, per cui i due metodi **evidenzia** e **rilascia** devono necessariamente avere un nome.

Il decoratore di metodo **@HostListener** fa sì che in corrispondenza del **mouseenter** sul componente ospite, venga eseguito il metodo di classe associato (cioè **evidenzia()**). Idem per il **mouseleave**

Esempi di utilizzo :

```
<a appHighlight >           // usiamo i valori di default
<a [appHighlight] >         // idem
<a appHighlight defaultColor = "LightYellow" highlightColor = "Yellow">
<a appHighlight [defaultColor]='LightYellow' [highlightColor]='Yellow' >
<a appHighlight highlightColor = "Yellow"> // uso solo highlightColor
```

Nel primo esempio Il tag **<a>** avrà sfondo **LightCyan** a riposo, e **Cyan** in corrispondenza dell'**hover**.

Nota 1

E' anche possibile assegnare ad un parametro di Input (ad esempio **highlightColor**) lo stesso nome del selettore

```
@Input() appHighlight?: string = 'Cyan';
```

In questo caso il colore relativo al **mouseenter** dovrà essere letto direttamente ad **appHighlight**

```
@HostListener('mouseenter') evidenzia(event: Event) {
  this.backgroundColor = this.appHighlight;
}
```

E l'utilizzo sarà il seguente: ****

Nota 2

Le Direttive Personalizzate seguono le normali regole di Binding di un attributo HTML:

- Se a destra dell'uguale **non** si usa una variabile ma un valore diretto, se questo valore diretto è una stringa, tale stringa deve essere scritta utilizzando ulteriori apici interni.
Se invece il valore diretto è un numero oppure true/false, allora deve essere scritto direttamente all'interno della stringa principale.
- Sempre nel caso di valori diretti, si potrebbero omettere le parentesi quadre sul nome della Direttiva e scrivere il valore di destra in forma diretta senza avvolgerlo in una ulteriore sottostringa
- Sorge però un problema: se l'elemento è di tipo **number** oppure **boolean** occorre necessariamente utilizzare le parentesi quadre perché:
defaultColor = 7 produrrebbe syntax error (a destra ci vuole sempre una stringa)
defaultColor = "7" assegnerebbe la stringa "7" invece del number 7
Per cui, alla fine, è consigliato usare **sempre** le quadre sui valori della direttiva

Direttive di Classe:

```
export class DropdownDirective {
  @HostBinding('class.open') isOpen: boolean = false;
  @HostListener('click') toggleOpen(event: Event) {
    this.isOpen = !this.isOpen;
  }
}
```

Le Direttive di Classe di solito non hanno parametri in **Input ()** da parte del componente ospite.

Non avendo parametri in input dovranno essere scritte all'interno del componente ospite senza parentesi quadre.

- All'interno della Direttiva viene definita una unica variabile booleana **isOpen** inizialmente false 'collegata' alla classe 'open' del componente ospite.
'open' è una classe **bootstrap** che quando viene applicata ad un drop down menù lo apre, mentre quando non viene applicata il menù si chiude
- In corrispondenza del click sul componente ospite viene richiamata la funzione **toggleOpen** (nome assolutamente arbitrario) che inverte il valore della variabile **isOpen**. Quando **isOpen** diventa true la classe bootstrap **open** viene applicata al componente ospite ed il menù viene aperto e viceversa.

Questa Direttiva Personalizzata può essere utilizzata per aprire/chiedere qualsiasi drop down menù bootstrap. Esempio di utilizzo all'interno di un componente ospite:

```
<ul class="nav navbar-nav navbar-right">
  <li class="dropdown" appDropdown>
    <a style="cursor:pointer;" class="dropdown-toggle" role="button">
      Manage <span class="caret"></span>
    </a>
    <ul class="dropdown-menu">
      <li </li> <li> </li>
    </ul>
  </li>
```

ngForm e la Validazione dell'input

Angular mette a disposizione uno strumento detto **ngForm** per la validazione degli input alternativo al controllo automatico di HTML5, cioè quello tale per cui, nel momento in cui inserisco in un textbox un valore che non soddisfa una certa regex, il browser automaticamente visualizza un suo messaggio di errore predefinito e blocca il submit (che però non interessa se si usano le chiamate ajax).

Per poter utilizzare **ngForm** occorre eseguire l'import di **FormsModule**

Occorre quindi inserire un tag **<form>** con l'attributo html **novalidate** in modo da inibire la validazione html del form e non interferisca con la validazione angular.

```
<form #loginForm="ngForm" novalidate>
```

Insieme alla form viene creato un oggetto nascosto **ngForm** che contiene un vettore con i puntatori a tutti i controlli presenti nella form, ognuno rappresentato mediante un oggetto **NgModel**

#loginForm rappresenta un reference a questo oggetto nascosto

Validazione di un form

Quando si applica [(**ngModel**)] ad un controllo inserito in una form come la precedente, Angular crea per ogni controllo un oggetto **ngModel** (istanza di **NgModel**) che rappresenta il singolo controllo presente nella form e contiene i seguenti stati del controllo:

- **touched** // diventa true dopo che il controllo è stato cliccato almeno una volta. Poi rimane true
- **valid** // se l'utente ha inserito un valore valido
- **invalid** // se l'utente ha inserito un valore non valido (required e campo vuoto o regex nok)
- **dirty** // true quando il contenuto del controllo è stato modificato rispetto al valore iniziale
- **errors** // riassuntivo per indicare quale errore si è verificato Es: **if(obj.errors.required)**

Per ogni controllo, all'interno dell'html è possibile definire un puntatore al controllo stesso, puntatore che in genere viene usato soltanto nell'html per gestire la grafica

```
[ (ngModel) ]="txtUsername"  
#txtUsernameCtrl = "ngModel" // reference al singolo controllo
```

A questo punto il pulsante di login potrà assumere il seguente aspetto, in cui passo alla procedura il reference all'intero oggetto di controllo della form, contenente tutti i singoli puntatori ai vari controlli.

```
<button type="button" (click)="eseguiLogin(loginForm)">  
    Login  
</button>
```

La procedura **eseguiLogin()** in caso di errori, segna tutti i controlli come "touched" in modo da ricolorare di rosso anche controlli che l'utente non ha ancora 'toccato'

Se tutto ok, invia la richiesta di login.

```
eseguiLogin(loginForm:any) {  
    if (loginForm.invalid) {  
        Object.values(loginForm.controls).forEach((control: any) => {  
            control.markAsTouched();  
        });  
        return;  
    }  
    alert('Login valido');  
    //this.eseguiRichiestaAjax();  
}
```

I servizi

Un servizio è una semplice classe che implementa delle funzionalità e/o gestisce dei dati che possono essere condivisi fra tutti i vari componenti che costituiscono l'applicazione.

Un servizio, oltre che essere utilizzato dai componenti, può, a sua volta, utilizzare le funzionalità fornite da altri servizi. Nelle prime versioni il servizio era indicato con il termine *provider*

In generale è buona regola sollevare i componenti dagli incarichi di logica "business" concentrandola invece all'interno dei Servizi. Un servizio è una classe indipendente dalle View che viene definita per svolgere un compito ben preciso nel rispetto del principio di singola responsabilità: ogni servizio si occupa di svolgere un unico e ben preciso compito.

Un tipico esempio di servizio è quello per l'invio di richieste Ajax ad un REST server.

Il service può essere creato manualmente come semplice classe oppure può essere autogenerato tramite CLI fermando il server ed utilizzando il seguente comando:

```
ng generate service services/recipe --skip-tests
```

Fino alla versione 18 in fase di creazione Angular aggiungeva automaticamente al servizio una desinenza **Service** abbastanza esplicativa. Volendo la si può aggiungere in fase di creazione:

```
ng generate service services/recipeService --skip-tests
```

All'interno della classe viene aggiunto un decoratore **@Injectable()** che rende il servizio **injectable**, cioè rende l'intera classe iniettabile al costruttore di un qualsiasi altro componente o servizio. Questo meccanismo per cui i servizi possono iniettarsi uno dentro l'altro è detto Dependency Injection ed è comodo per evitare di registrare tutti i vari servizi in forma centralizzata.

Riassumendo il comando precedente crea un servizio:

- che si chiama **RecipeService** (con l'iniziale maiuscola, di solito impostato al singolare)
- con applicato un decoratore **@Injectable()** così strutturato :
@Injectable({providedIn:'root'})
dove **providedIn:'root'** significa che il servizio viene creato all'inizio e poi è utilizzabile in qualsiasi componente (una specie di singleton).
Mettendo invece 'recipe-component' sarebbe utilizzabile solo all'interno del componente Recipe
- Sui Service l'evento **onInit()** non è significativo e non viene generato.
- Eventuali inizializzazioni possono essere eseguite all'interno del costruttore.

Tutti i componenti a cui verrà iniettato il servizio avranno accesso condiviso a Proprietà e Metodi del servizio medesimo. .

Attenzione però che nel costruttore di un servizio non è consentito lanciare subito un metodo di richiesta dati, richiesta che deve necessariamente essere avviata da un componente. Un servizio infatti viene istanziato nel momento in cui viene istanziato il primo componente che lo referencia.

Creazione di un servizio base per l'invio delle richieste Ajax

Si vuole creare un servizio denominato **DataStorageService** che si occupi di inviare tutte le chiamate http verso un server remoto.

```
ng generate service shared/dataStorageService --skip-tests
```

Il modulo angular che si occupa di gestire le chiamate http è l'oggetto **HttpClient**

Dalla versione 18 in avanti è stato aggiunto un file **app.config.ts** che sopprime in parte al file app.module ed in cui occorre dichiarare esplicitamente l'utilizzo dell'oggetto HttpClient:

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners,
provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
import { provideHttpClient, withFetch } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withFetch()),
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes)
  ]
};
```

L'opzione withFetch() fa sì che HttpClient utilizzi fetch() invece che XMLHttpRequest (default).

Impostazione di un DataStorageService con interfaccia CRUD

Occorre innanzitutto importare l'oggetto **HttpClient**

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { Observable } from 'rxjs';
```

Per iniettare l'oggetto **HttpClient** all'interno della classe corrente ci sono 2 possibili sintassi:

- nuova, preferibile
private httpClient: HttpClient = inject(HttpClient);
- vecchia ma ancora supportata
constructor(private httpClient: HttpClient) { }

Indirizzo del server : **private REST_API_SERVER = "http://localhost:3000/api"**

I vari metodi dell'oggetto httpClient si aspettano 2 parametri:

- una URL (con in coda l'eventuale ID del record da ricercare)
- un eventuale un json di parametri aggiuntivi.

Indipendentemente dal tipo di chiamata :

- I parametri GET possono essere accodati alla URL in formato **URL-encoded** oppure inseriti come JSON all'interno di una apposita chiave **params**. In quest'ultimo caso provvederà Angular a convertirli in **URL-encoded** e concatenati alla URL dopo il ?
{"params":{"nome":"pippo", etc}}
- Nel caso dei parametri **body** occorre fare una distinzione:
 - a) Nelle chiamate POST, PATCH, PUT lo standard http prevede il passaggio diretto dei parametri all'interno del body della http request, per cui possono essere passati ad httpClient direttamente come secondo parametro **{"nome":"pippo", etc}**
 - b) Nel caso invece delle chiamate DELETE lo standard http non prevede il passaggio diretto dei parametri all'interno del body della http request, anche se molti server lo supportano, ma prevede invece il passaggio degli eventuali parametri in **URL-encoded**
httpClient si aspetta come secondo parametro una **option** in cui occorre specificare se si intende passare gli eventuali parametri come URL-encoded oppure nel body della request.
Quindi come secondo parametro si può passare
{"params":{"nome":"pippo", etc}} // oppure
{"body":{"nome":"pippo", etc}}

La prima sintassi è più standard, ma anche la seconda è quasi sempre riconosciuta

```

public inviaRichiesta( method:string, resource:string, params:any={} )
:Observable<any> | undefined {
  resource = this.REST_API_SERVER + resource
  switch(method.toLowerCase()) {
    case "get":
      return this.httpClient.get(resource, {"params":params})
    case "post":
      return this.httpClient.post(resource, params)
    case "patch":
      return this.httpClient.patch(resource, params)
    case "put":
      return this.httpClient.put(resource, params)
    case "delete":
      return this.httpClient.delete(resource, {"params":params})
    default: return undefined; } }

```

- Riguardo alla patch, se si accoda l'ID alla URL, json-server esegue automaticamente SEMPRE una operazione di tipo `$set` dei parametri ricevuti (**simple patch**). Idem il nostro crud-server
- Riguardo alla put, nel crud-server è stato implementato un **complex patch** in cui è possibile specificare l'intera operazione da eseguire sull'ID accodato alla risorsa:


```
{ "$inc": {"nVampires":1} }
```
- Siccome il default case del precedente switch restituisce undefined, chi si mette in ascolto dell'Observable tramite subscribe(), dovrà sempre avere il ?:
 this.dataStorageService.inviaRichiesta("GET", '/recipes/' + id)?.subscribe()

Accesso ai dati ricevuti : gli oggetti Observable, Observer e Subscriber

Tutti i metodi dell'oggetto `HttpClient` restituiscono al chiamante un oggetto **Observable** che è sostanzialmente un **produttore di dati**. E' un oggetto analogo all'oggetto `Promise` di javascript, ma con maggiore flessibilità ed anche maggiore potenzialità. E' in corso una proposta di standardizzazione dell'oggetto Observable anche in javascript (proposta TC39).

L'oggetto **Observable** rende disponibile un metodo `.subscribe()` che si aspetta come parametro un oggetto **Observer** che è un json costituito da 3 chiavi :

- **"next"** che ha come valore una funzione a cui vengono iniettati i dati prodotti dall'Observable
- **"error"** che ha come valore una funzione a cui viene iniettato un oggetto `err` contenente varie informazioni relative all'errore.
- **"complete"** che ha come valore una funzione richiamata sia dopo il next che dopo l'error.

In pratica l'**Observable** produce i dati, mentre il **Subscriber** creato mediante il metodo `subscribe()` si mette in ascolto dei dati e li consuma attraverso l'oggetto **Observer** ricevuto come parametro.

Esempio di utilizzo di un servizio all'interno di un componente

```

constructor(public dataStorageService: DataStorageService) { }

getRecipes() {
  let observable:Observable<any> =
    this.dataStorageService.inviaRichiesta('get', '/recipes')?.subscribe({
      "next" : (data) => {
        this.recipes = data;
        this.selectedRecipe = this.recipes[0];
      },
      "error" : (error:any) => { console.log(error) }
    })
}

```


Volendo utilizzare una variabile intermedia di tipo `Observable` occorre importare l'oggetto da `'rxjs'`

```
import { Observable } from 'rxjs';
```

Un servizio iniettato ad un costruttore può assumere i seguenti qualificatori di visibilità:

- **public** se proprietà e metodi del servizio devono essere visibili anche all'interno del file html
- **private** in caso contrario (visibili soltanto all'interno della classe)

I metodi appartenenti ad un servizio iniettato:

NON devono essere utilizzati all'interno del costruttore del componente ,

ma è **preferibile** utilizzarli all'interno di `ngOnInit(): void { }` del componente

- **NON** possono essere utilizzati all'interno del metodo `onInit()` di `AppComponent`, perché `AppComponent` viene istanziato *prima* che i servizi siano stati avviati.
- **Espone** dei metodi ma **non li esegue**. A richiamare i metodi saranno sempre i componenti.

Utilizzo delle Arrow Function all'interno di subscribe

Per la scrittura delle funzioni di callback occorre utilizzare le arrow function perché, lavorando sempre all'interno di metodi di classe, **nel caso delle arrow function il `this` mantiene il contesto a monte rispetto alle funzioni stesse, cioè continua sempre a puntare alla classe corrente.**

Viceversa se si utilizzano per le callback le normali funzioni "tradizionali", al loro interno il `this` rappresenterebbe l'oggetto `Promise / Observable` all'interno del quale si sta operando.

Nota sugli errori

In caso di errore `HttpClient` restituisce un oggetto **`err`** contenente diversi campi, fra cui:

- `status` codice di errore (es 401)
- `statusText` Messaggio di errore predefinito (es Unauthorized)
- `message` Messaggio del tipo Http failure response for https://localhost:3001/api/login: 401 Unauthorized
- `error` Messaggio di errore specifico costruito dal server

Il controllo dell'applicazione tramite i Services

Invece di inviare le request all'onLoad del componente, si può creare un ulteriore **servizio intermedio** che, mediante il `DataService` precedente, **si occupi di inviare le richieste http e di salvare i dati al suo interno**, dati che potranno poi essere utilizzati da qualsiasi componente. In questo modo i componenti si occuperanno soltanto più della visualizzazione dei dati. Semplice, veloce, snello.

Concetto di **Controller** tipico del paradigma **MVC**:

- Model** -> Struttura dei dati (es ricette e ingredienti)
- View** Rappresentazione dei dati attraverso i **Componenti**
- Controller** Gestione centralizzata della "business logic" attraverso i **Servizi**

Per ogni componente di livello più alto dell'applicazione (le cosiddette **features**) si va a creare un relativo servizio che ne memorizza i dati.

es: StudentService

All'interno del costruttore si inietta il `DataService` precedente

```
constructor(private dataStorageService: DataService) { }
```


Dopo di che si possono definire tutti i metodi relativi ad esempio alla gestione degli **students**:

- `getStudents()` richiederà al server l'elenco completo di tutti gli studenti
- `getStudentById(id)` richiederà un singolo studente sulla base dell'ID
- `getStudentByCity(city)` richiederà tutti gli studenti di una certa città
- `getStudentByGenderAndClass(gender, class)` filtro su genere e classe

Gestione del subscribe

Il subscribe in genere dovrebbe essere eseguito **dal componente** il quale avrà così la possibilità di aggiornare delle label si stato sull'esito dell'operazione.

richieste POST, PATCH, DELETE

In questo caso il servizio potrebbe limitarsi a demandare il subscribe al chiamante ed eseguire un semplice return. Cioè il metodo `inviaRichiesta()` restituisce un **Observable** che tramite return rimandiamo al chiamante.

```
postRecipe(id:any, body:any) {  
    return this.dataStorageService.inviaRichiesta('post', '/recipes/'+id, body)  
}
```

In alternativa il servizio potrebbe eseguire il subscribe all'interno del servizio e, al termine, emettere un nuovo evento, in corrispondenza del quale il componente chiamante potrà visualizzare messaggi e errori all'interno della pagina html. Decisamente più complicato e comunque sconsigliato.

richieste GET

In questo caso sarebbe forse meglio eseguire il subscribe all'interno del servizio, in modo che il servizio possa salvare i dati al suo interno e tutti i componenti li possano poi utilizzare.

Il componente chiamante, subito dopo la chiamata potrà accedere ai dati che forniranno un feedback immediato all'utente. Notare che, mentre il servizio è in ascolto sul subscribe in attesa dell'arrivo dei dati, il componente può comunque eseguire il codice html di visualizzazione dei dati anche senza aspettare il next. **Quando i dati arriveranno, Angular provvederà automaticamente ad aggiornare l'interfaccia grafica con i dati ricevuti.**

Per evitare errori durante l'attesa dei dati, è sufficiente concatenare il **?** in coda alla variabile. In questo modo, in caso di undefined, semplicemente i corrispondenti elementi html non verranno visualizzati.

Intercettazione dei dati all'interno del servizio

Il fatto che il subscribe venga eseguito all'interno del servizio non è una buona strategia (bad practice) perché il chiamante, pur vedendo i dati, non ha modo di vedere e gestire degli errori.

A tale scopo, *anche nel caso delle chiamate GET*, il servizio potrebbe eseguire semplicemente un return demandando il subscribe al chiamante.

L'oggetto **Observable** dispone di un comodissimo metodo **pipe()** che consente di applicare degli operatori all'Observable stesso. In parole semplici rappresenta un "tubo" dentro cui far passare i dati per trasformarli o intercettarli prima del subscribe().

```
import { tap } from 'rxjs';

getRecipe(id:any){
  return this.dataStorageService.inviaRichiesta('get', '/recipes/'+id)
    .pipe(
      tap((data) => {
        this.dataList = data;
      })
    );
}
```

Il metodo **tap()** consente di elaborare i dati che 'passano' dentro pipe e di salvarli all'interno di una variabile locale del servizio stesso.

Il componente eseguirà normalmente il subscribe senza doversi preoccupare di leggerne e salvare i dati

Conversione di un Observable in Promise

L'oggetto **Observable** di TypeScript può facilmente essere convertito in una **Promise** JavaScript facendo semplicemente uso del metodo **.toPromise()** peraltro deprecato nel 2023.

Notare che **async / await** è un costrutto utilizzabile esclusivamente con le Promise, e NON con gli Observable, per cui se si desidera utilizzare questo costrutto occorre necessariamente convertire l'Observable in Promise.

Il codice precedente potrebbe essere riscritto nel modo seguente:

```
let promise=this.dataStorageService.inviaRichiesta('get', '/recipes').toPromise();
promise.then( (data: RecipeModel[]) => {
  this.recipes = data;
})
promise.catch( (error: any) => {
  console.log(error);
})
```

oppure nel modo seguente, senza far uso del metodo **.toPromise()**

```
getRecipes(): Promise<any> {
  return new Promise((resolve, reject) => {
    this.http.get('/recipes').subscribe({
      next: (data) => resolve(data),
      error: (err) => reject(err),
    });
  });
}
```

Conversione di una Promise in Observable

```
import { Observable, from } from 'rxjs';

let myObservable = from(myPromise);
```

Generazione di eventi tramite Subject

Per generare un evento si potrebbe utilizzare un oggetto **EventEmitter**. La documentazione raccomanda però di utilizzare questo evento solo ed esclusivamente in abbinamento al decoratore `@Output` per passare dei parametri da un Bottom Component ad un Top Component.

Nel caso dei servizi è raccomandato di utilizzare l'oggetto base **Subject** che è un oggetto più leggero da cui eredita **EventEmitter** e che presenta la stessa sintassi. L'unica differenza è che per emettere l'evento NON si usa `.emit()` ma `.next()`

Ad esempio l'aggiunta di un nuovo documento all'interno di una collezione potrebbe essere gestito nel modo seguente:

```
ingredientAdded: Subject<string> = new Subject<string>();

postIngredient(ingredient: IngredientModel) {
  this.storageService.inviaRichiesta('post', '/shopping-list', ingredient)
    ?.subscribe({
      next: (data) => {
        console.log(data);
        this.ingredientAdded.next('Ingredient successfully added!');
      },
      error: (error: any) => {
        console.log(error);
      },
    });
}
```

Il componente chiamante, nell'`ngOnInit()`, si metterà in ascolto dell'evento **ingredientAdded**

```
ngOnInit() {
  let subscriber = this.shoppingListService.ingredientAdded
    .subscribe((param: string) => {
      alert(param)
    })
}
```

Rilascio del Subscriber

Gli oggetti **Subject** ed **EventEmitter**, al termine delle operazioni, non rilasciano l'oggetto **subscriber** istanziato dal `subscribe()`, il quale dunque rimarrà in ascolto all'infinito, o per lo meno fino a quando non viene deistanziato l'oggetto **Observable** fornitore dei dati.

Nell'esempio precedente, l'oggetto **Subject** istanziato all'avvio del servizio rimane attivo fino a quando non viene deistanziato il servizio stesso. Poiché i servizi non vengono sostanzialmente MAI deistanziati, il **subscriber** precedente rimarrà in ascolto all'infinito. Non solo, ma ogni volta che il componente viene nuovamente visualizzato, rigenera l'evento `ngOnInit()` il quale genera un nuovo **subscriber** e di conseguenza genera un nuovo **Observer** di ascolto. Per cui, dopo un po', l'evento (e la sua `alert`) verranno eseguiti un numero imprecisato di volte.

La soluzione a questo problema consiste nel rilasciare il **subscriber** all'interno dell'evento opposto a `ngOnInit()`, cioè `ngOnDestroy()` :

```
ngOnDestroy() {
  this.subscriber.unsubscribe()
}
```

All'interno del `postIngredient()` precedente si potrebbe ancora ipotizzare il seguente miglioramento:

```
if(!this.ingredientAdded.observeds) // se NON ci sono listener
  alert("Ingredient successfully added!");
else
  this.ingredientAdded.next('Ingredient successfully added!');
```

Routing delle pagine

In una applicazione Web, per "routing" si intende il meccanismo che consente la navigazione all'interno dei vari componenti che costituiscono l'applicazione.

- Nelle applicazioni **multi-page**, il routing viene realizzato molto semplicemente cambiando pagina. Sorge però il problema di dover passare i parametri da una pagina all'altra.
- Le applicazioni **single-page** sono più semplici da gestire però, se l'applicazione gestisce contenuti multipli, tutte le ricerche punteranno alla stessa pagina senza alcuna differenziazione.
- Il routing **unisce le due cose** facendo in modo che una applicazione single page diventi visibile come se fosse multi-page. Cioè i vari componenti verranno visti dall'esterno come "pagine" **INDIPENDENTI** ciascuna con una propria URL fittizia e, di conseguenza, VIENE MIGLIORATA l'indicizzazione da parte dei motori di ricerca che potranno puntare alla pagina corretta.
- Inoltre, mediante le cosiddette routes parametriche, ad esempio sulla base di un ID, diventa anche possibile vedere i singoli dettagli di un elemento come fosse una pagina indipendente.

Per la gestione del routing, Angular fornisce un modulo apposito chiamato **Router** che gestisce il routing tramite la definizione di appositi oggetti **Routes**.

Scrittura delle Routes

Le routes devono essere scritte **senza lo slash** e verranno automaticamente concatenate al path di dominio della pagina corrente.

Da codice però, in fase di chiamata, occorre anteporre lo slash davanti. Ad es **/recipes**

```
export const routes : Routes = [  
  { path: '',  
    redirectTo: '/recipes',  
    pathMatch: 'full'  
  },  
  { path: 'recipes',          // senza slash !  
    component: RecipesComponent,  
  },  
  { path: 'shopping-list',    // senza slash !  
    component: ShoppingListComponent  
  }  
];
```

- Le routes possono essere di due tipi:
 - routes di **esecuzione di un redirect**.
 - routes per il **caricamento di un component**,
- Il path iniziale (vuoto) è un path di **redirect** ed indica cosa visualizzare quando l'utente accede al sito senza specificare nessuna risorsa. Nel nostro caso vogliamo eseguire un redirect al path **/recipes**. L'opzione **pathMatch: 'full'** è obbligatoria e sta ad indicare che il match deve essere eseguito per intero, cioè deve andare a buon fine SOLO quando l'utente inserisce veramente stringa vuota. Al posto di **redirectTo** e **pathMatch** si poteva impostare **component: RecipesComponent**, nel qual caso verrebbe caricato sempre **RecipesComponent**, **però NON verrebbe aggiornata la URL all'interno della barra di navigazione**.

Struttura di AppRoutingModuleModule in Angular 16

Quando si crea una nuova applicazione Angular 16, se si utilizza il flag **--routing** automaticamente Angular crea il modulo **AppRoutingModule** all'interno di un apposito file **app-routes.ts**

```
ng new my-app --routing
```

Se invece l'applicazione è già stata creata è comunque possibile creare **AppRoutingModule** tramite il comando: `ng generate module app-routing --flat --module=app`

`--flat` puts the file in `src/app` instead of its own folder.

`--module=app` tells the CLI to register it in the `imports` array of the `AppModule`.

in pratica aggiunge `AppRoutingModule` fra gli imports del file `app.module.ts`

```
imports: [BrowserModule, FormsModule, AppRoutingModule ],
```

Dopo aver creato l' **AppRoutingModule**, in alto occorre eseguire l'import delle seguenti due librerie:

```
import { Routes, RouterModule } from '@angular/router';
```

Occorre quindi definire all'interno del file il vettore delle **Routes []** in cui si definiscono tutte le varie **routes** necessarie all'applicazione, come indicato nella pagina precedente.

Nella parte finale di **AppRoutingModule** viene automaticamente inserito il decoratore **@NgModule** che riceve un oggetto di inizializzazione che dovrebbe essere impostato nel modo seguente, con due righe "fisse" che servono sostanzialmente ad inizializzare il router. `RouterModule` è il modulo importato all'inizio del quale si invoca il metodo `.forRoot ()` passandogli come parametro il vettore delle routes

```
@NgModule ({
  declarations: [], // per definire eventuali variabili globali
  imports: [RouterModule.forRoot( appRoutes )],
  exports: [RouterModule],
})
```

Il Routing in Angular 17

In Angular 17 non esiste più il file **AppRoutingModule** ma al suo posto:

- Viene creato un file **app.routes.ts** contenente soltanto il vettore delle **Routes** necessarie all'applicazione, come indicato nella pagina precedente.

```
export const routes: Routes = [ ]
```

- All'interno di **app.component.ts** importare **RouterOutlet**
- In tutti i componenti che utilizzano **RouterLink** e **RouterLinkActive**, occorre importarli :

```
@Component ({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, FormsModule, RouterLink, RouterLinkActive],
})
```

Navigazione delle routes da html e da codice: l'oggetto <router-outlet>

Nella sezione principale della APP (**app-component**) si inseriscono tipicamente due oggetti :

- una **nav-bar** di navigazione per la visualizzazione dei componenti dentro **<router-outlet>**
- un componente **<router-outlet>** che funge da **visualizzatore** per i vari componenti selezionati tramite `RouterLink` all'interno della parte fissa

```
<app-header> </app-header>
<router-outlet> </router-outlet>
```

Ogni volta che da html o da codice si imposta una nuova route, angular provvederà a visualizzare all'interno di <router-outlet> il componente associato a quella route.

Per impostare una nuova route ci sono tre possibilità:

- 1) Richiedere la route direttamente dalla barra di navigazione
`http://localhost:4200/shopping-list`
- 2) Utilizzare l'attributo **routerLink** all'interno del file .html. L'attributo **routerLink** è simile ad **href** e può essere utilizzato sostanzialmente su quasi tutti i tag (a patto di non gestire l'evento **click**). Nella **nav-bar** si possono inserire i vari link per caricare i relativi componenti nel `<router-outlet>`

```
<li> <a routerLink="/"> Home </a> </li> // ricarica la pagina
<li> <a routerLink="/recipes">Recipes</a> </li>
<li> <a routerLink="/recipes"> Recipes</a> </li>
<li> <a [routerLink]="['/shopping-list', '2']">Shopping List</a> </li>
```

- Lo **slash** iniziale è **obbligatorio** per far capire che si tratta di una route assoluta.
- Il valore della url **può** essere passato come valore diretto oppure come variabile nel qual caso occorre scrivere `[routerLink]` all'interno di parentesi quadre.
- Se si scrive `[routerLink]` all'interno di parentesi quadre, **a destra diventa obbligatorio utilizzare un vettore di stringhe** (anche se la stringa è una sola). Queste stringhe saranno poi automaticamente concatenate per comporre la url finale. Gli **slash** intermedi vengono aggiunti automaticamente come elemento separatore fra le stringhe.
- Prestare attenzione a rimuovere da tutti i tag `<a>` l'attributo `href="#"` perché altrimenti, al click, viene ricaricata la pagina. Aggiungere poi nel CSS `a {cursor:pointer}`

Attenzione che **routerLink non è disponibile sui controlli (tag input e select)** perché **Angular non associa automaticamente routerLink al cambio di stato di un controllo**.

Per cui se si vuole associare un menù di navigazione a dei radio buttons o listbox, occorre gestire la navigazione da codice.

- 3) La terza alternativa consiste appunto nel gestire un evento `click()` (disponibile su qualunque tag) e poi caricare la pagina da codice utilizzando il metodo **router.navigate()**. A tal fine occorre iniettare al costruttore il componente Injectable **Router**:

```
<li> <a (click)="caricaRicette()"> Recipes </a> </li>
constructor(private router:Router) { }
caricaRicette(id:string){
  this.router.navigate(['/recipes'])
}
```

 - Il componente **router**, se **public**, può essere utilizzato direttamente all'interno del `(click)` nel file html.
 - Nel caso di `navigate()` il primo slash deve essere **inserito** solo nel caso dei percorsi assoluti
 - `router.navigate()` si aspetta **SEMPRE** come parametro un vettore di stringhe che verranno poi concatenate per comporre la URL.
 - **In sostanza la procedura caricaRicette, naviga verso un sottocomponente (/recipes) il quale sarà lui a richiamare l'elenco di tutte le ricette.** Più flessibile perché in questo modo diventa possibile richiamare le ricette anche dalla barra di navigazione.
 - Attenzione però che `router.navigate` e `routerLink` producono effetto SOLO quando ricevono una URL diversa dalla URL corrente. Se si esegue un nuovo `router.navigate` verso la URL attualmente già visualizzata, il nuovo `navigate` NON viene eseguito, per cui NON può essere sfruttato per aggiornare in qualche modo i dati del componente attualmente visualizzato.
 - Se in corrispondenza del refresh della pagina si desidera visualizzare un certo componente iniziale, si può richiamare il metodo `navigate()` nell'`onInit()` di `<app-header>` o `<app-component>`

```
this.router.navigate(['/'])
```

L'attributo routerLinkActive

La Direttiva **routerLink** dispone di un ulteriore comodissimo attributo **routerLinkActive** che, applicato a tutte le voci di un menù, consente di applicare automaticamente una certa classe alla voce attualmente selezionata. Ad esempio si può utilizzare questo attributo per aggiungere la classe bootstrap **active** alla voce di menù attualmente selezionata (che però, nel caso di bootstrap, va aggiunta sull'LI esterno).

```
<li routerLinkActive="active"><a routerLink="/recipes">Recipes</a></li>
<li routerLinkActive="active"><a routerLink="/shopping-list">ShList</a></li>
```

Attenzione che anche **routerLinkActive** deve essere annoverato fra gli import.

Inoltre **routerLinkActive** funziona soltanto per quei link definiti via html attraverso la Direttiva routerLink e NON per le sezioni eseguite da codice mediante il metodo router.navigate().

Routes parametriche

Si supponga di voler visualizzare sempre all'interno della pagina principale (cioè all'interno del medesimo **<router-outlet>** di **app.component**) i dettagli di una specifica ricetta, ad esempio la ricetta avente id = 2

Si desidera inoltre che la pagina sia richiamabile anche dalla barra di navigazione scrivendo

```
http://localhost:4200/2
```

A tale scopo si può aggiungere nel vettore delle **routes** la seguente **route parametrica** introdotta dai due punti:

```
{ path: ':id', component: DettagliRicettaComponent }
```

Nel momento in cui si definisce una route parametrica, **qualunque richiesta che NON soddisfi alle altre routes** presenti nel vettore (nel nostro caso **/recipes** e **/shopping-list**) **verrà automaticamente intercettata dalla route parametrica**.

Per cui ad ogni livello dell'albero delle routes ci può essere sempre solo una unica route parametrica che deve necessariamente essere scritta **sempre** per ultima fra le routes di pari livello e che intercetterà tutte le richieste non soddisfatte nelle routes antecedenti.

L'utente potrà richiamare la route parametrica digitando qualunque informazione, numerica o alfanumerica, che verrà comunque sempre interpretata come stringa

Le seguenti chiamate verranno TUTTE intercettate dalla stessa route parametrica precedente:

```
http://localhost:4200/2
http://localhost:4200/ricetta34
http://localhost:4200/qualunqueAltraCosa
```

La route parametrica potrà essere richiamata nei soliti modi :

```
<li><a routerLink = "2"> Dettagli ricetta 2</a></li>
<li><a [routerLink]="[id]">Dettagli ricetta</a></li>
caricaDettagli() {
  this.router.navigate(['2'])
}
```


Lettura dei parametri

Nel caso delle routes parametriche, **la visualizzazione del componente non è automatica**, perché dipende strettamente dal ID che è stato impostato. Per cui il componente `Dettagli`, richiamato in corrispondenza del click su un link, dovrà accedere all'ID della ricetta richiesta e richiedere al server le informazioni corrispondenti.

Per poter accedere ai parametri ricevuti da una route parametrica, occorre innanzitutto iniettare al costruttore l'oggetto `ActivatedRoute` contenente la route attualmente selezionata:

```
constructor( private activatedRoute:ActivatedRoute) {}
```

L'oggetto `ActivatedRoute` genera un `Observable params` **ogni volta che viene attivata una qualsiasi route associata a quel componente** (parametrica o non parametrica).

In caso di route parametrica (che inizia con i due punti) alla funzione di gestione dell'evento viene iniettato un json contenente il parametro di attivazione della route, con il nome parametrico utilizzato all'interno del file di routing. Per cui, in questo caso, il json iniettato al subscribe sarà `{ "id":2 }`

Ascolto dell'evento

Il componente, tramite il metodo `.subscribe()`, dovrà mettersi in ascolto dell'`Observable params` generato ogni volta che una route porterà al caricamento del componente attuale.

L'associazione tra l'evento `subscribe()` e la procedura di gestione dell'evento viene generalmente eseguita in fase di visualizzazione del componente, cioè all'interno del metodo `onInit()`

```
ngOnInit(): void {
  this.activatedRoute.params.subscribe(
    (params: Params) => {
      if (params['id']) {
        console.log(params['id'])
        this.recipeService.getRecipe(params['id']);
      }
    }
  )
}
```

Nota

L'`Observable params` viene generato **anche in corrispondenza dell'impostazione della sua route dalla barra di navigazione**. Estremamente comodo come in una vera applicazione multipage.

Soluzione alternativa

Ancora più semplicemente si potrebbe utilizzare il seguente metodo sincrono:

```
let detailsId = this.route.snapshot.paramMap.get("id");
```

Notare però che inserire questo metodo sincrono all'interno di `onInit()` non è equivalente alla soluzione precedente. Infatti la riga precedente verrebbe eseguita soltanto in corrispondenza dell'avvio. Nel momento in cui si passa da una route parametrica all'altra (ad esempio da `/user/1` a `/user/2`) l'evento `onInit()` **NON viene generato** e dunque l'aggiornamento della ricetta non viene eseguito. Viceversa il `subscribe()` si mette in ascolto dell'evento `params()` che viene generato in corrispondenza di ogni variazione della route parametrica.

Child routes

Riprendendo l'esempio precedente, il fatto di posizionare la route parametrica al primo livello non è la soluzione migliore. Sarebbe meglio impostare la route parametrica come sotto-route di `recipes`, in modo che l'utente possa richiamare i dettagli di una ricetta nel modo seguente:

```
http://localhost:4200/recipes/2
```

Una route di primo livello, tramite l'attributo **children** (che ha la forma d un vettore enumerativo), può esporre una o più **child route**, assegnando a ciascuna un corrispondente componente.

```
{
  path: 'recipes', component: RecipesComponent,
  children: [
    { path: '', component: RecipeStartComponent },
    { path: 'new', component: RecipeEditComponent },
    { path: ':id', component: RecipeDetailComponent },
    { path: ':id/edit', component: RecipeEditComponent }
  ]
}
```

- **path: 'new'**, in corrispondenza della route **recipe/new**, carica un componente per l'inserimento di una nuova ricetta (che nell'esempio coincide con quello di editazione)
- **path: ':id'**, in corrispondenza della route **recipe/2** carica il componente **recipe-detail** per la visualizzazione dei dettagli della ricetta indicata.
- **path: ''** interviene nel momento in cui l'utente richiede **/recipes** senza specificare nessun id, nel qual caso caricherà nell'area di destra un messaggio `<p>Please select a Recipe!</p>`
- **path: ':id/edit'** caricherà un componente per l'editazione della ricetta corrente, precaricando tutti i valori correnti.

Richiamo delle child-routes

Le child routes relative ai sottocomponenti possono essere richiamate da **html** o da **TypeScript** al solito modo:

HTML

```
<a [routerLink]="['new']"> </a>
<a [routerLink]="[recipe.id]"> </a>
```

Notare che, a differenza delle routes di primo livello, **le child routes passate a routerLink NON devono iniziare con lo slash**, proprio perchè sono routes relative che verranno automaticamente concatenate alla route corrente. Il componente contenente questo link è un componente che sarà visualizzato soltanto in corrispondenza della route **/recipes**.

In corrispondenza del click, Angular provvederà a concatenare il valore di **routerLink** alla route corrente, producendo come risultato ad esempio **/recipes/2**

Type Script

Usando il metodo `navigate()` il concatenamento precedente non è automatico ma occorre specificare un secondo parametro che indichi a quale route deve essere concatenato il "path relativo" contenuto nel primo parametro. Il valore del secondo parametro sarà praticamente sempre **this.route**

```
constructor(private router:Router, private activatedRoute:ActivatedRoute){}
onNewRecipe() {
  this.router.navigate(['/recipes', 'new']) // oppure
  this.router.navigateByURL('/recipes/new') // oppure
  this.router.navigate(['new'], { relativeTo: this.route })
}
```

Al solito, con **navigate()**, è obbligatorio l'uso del vettore.

new, essendo una child-route, **DEVE** essere scritta senza slash. Viceversa lo **slash** davanti a **recipes** deve essere inserito.

In caso di click sui dettagli della ricetta corrente **/recipes/2** andiamo a concatenare **"edit"** per il caricamento del componente preposto all'editazione del record corrente

```
this.router.navigate(['edit'], { relativeTo: this.route })
```

Criterio di visualizzazione delle routes

Quando viene impostata una route mediante click sul un tag `<a>` oppure mediante il metodo `router.navigate()`, Angular in prima battuta verifica se si tratta di una **route assoluta** (`/recipes`) oppure di una **child-route** (`/recipes/new`)

- Se si tratta di una **route assoluta**, il componente associato alla **route** viene caricato all'interno del `<router-outlet>` di `app.component`, il quale deve sempre avere al suo interno un `<router-outlet>`. Se non ci fosse, semplicemente la route non viene 'servita'
- Se si tratta invece di una **child-route**, Angular guarda alla **Parent-Route** (`/recipes`). Se la **Parent-Route** (`/recipes`) ha un componente assegnato, questo componente dovrebbe esporre un `<router-outlet>`. **Ogni componente che ha delle child-routes deve sempre esporre un `<router-outlet>` in cui verranno visualizzati i componenti 'puntati' dalle child-routes**. Se il componente padre non espone un `<router-outlet>`, allora la child route non viene servita.

Per cui lo scenario precedente può essere completato assegnando a `RecipesComponent` un `<router-outlet>` in cui visualizzare **TUTTI** i sotto componenti associati alle varie **child-routes**.

Quindi, all'interno di `RecipesComponent`, si potranno definire due sezioni:

- Una sezione di sinistra contenente la lista delle ricette più alcuni pulsanti, ad esempio per l'inserimento di una nuova ricetta
- Una sezione di destra contenente il `<router-outlet>` in cui verranno visualizzati i dettagli della ricetta selezionata e, ad esempio, una form per l'inserimento di una nuova ricetta.

```
<div class="row">
  <div class="col-md-5">
    <app-recipe-list></app-recipe-list>
  </div>
  <div class="col-md-7">
    <router-outlet> </router-outlet>
  </div>
</div>
```

Casi Particolari

- Come detto, se la **Parent-Route** (`/recipes`) ha un componente assegnato, e questo componente non espone un `<router-outlet>`, allora la route non viene servita
- Se la **Parent-Route** (`/recipes`) **NON** ha un componente assegnato (e dunque non può esporre un `<router-outlet>`), allora Angular provvede a scorrere l'albero dei componenti verso l'alto finché non individua un selettore `<router-outlet>`, fino ad arrivare eventualmente ad `app.component`. Nel caso arrivi fino ad `app.component` sostituisce però completamente la Main Page attualmente visualizzata. Non va comunque bene.

A tal proposito, al di là del funzionamento, sorge spontanea una domanda: come si fa a dichiarare una route senza assegnargli un componente? Si usa un piccolo 'trucco' riportato di seguito:

<https://medium.com/dev-genius/the-art-of-nested-router-outlets-in-angular-dafb38245a30>

```
export const routes: Routes = [
  { path: '', redirectTo: 'mainpage', pathMatch: 'full' },
  { path: 'mainpage',
    children: [
      { path: '', component: MainpageComponent },
      { path: 'first', component: Tab1Component },
      { path: 'second', component: Tab2Component }
    ]
  }
]
```

`mainPage` non ha un componente associato ma viene visualizzata comunque grazie alla childroute vuota

Lazy Loading Modules

Per impostazione predefinita, i moduli Angular vengono caricati nel momento in cui l'app viene caricata, indipendentemente dal fatto che siano immediatamente necessari oppure no. Per app di grandi dimensioni con molti percorsi, è previsto un modello di progettazione che consente di caricare gli NgModule solo quando necessari. Il **Lazy Loading** (letteralmente caricamento lento) aiuta a mantenere le dimensioni dei bundle iniziali più piccole, il che a sua volta aiuta a ridurre i tempi di caricamento.

Per poter utilizzare il Lazy Loading di un certo componente (es. `home`) occorre creare non un semplice componente ma un NgModule utilizzando il seguente comando:

```
ng generate module home --route home --module app.module
```

comando che genera un componente `home` completo comprensivo di due files aggiuntivi:

```
home.module.ts
home-routing.module.ts
```

Il 1° file contiene l'import di tutti i moduli necessari all'esecuzione del componente. Eventuali moduli aggiuntivi dovranno essere inseriti in questo file e saranno caricati al momento del caricamento di questo modulo. In questo modo il file globale `app.module.ts`, contenente i files da caricare all'avvio, risulta molto più leggero e di rapida esecuzione.

La sezione `declarations: [ ]` dichiara ad Angular l'esistenza del componente.

Il 2° file contiene le informazioni di routing relative al file corrente ed è quasi sempre espresso nel seguente formato:

```
const routes: Routes = [
  {
    path: '',
    component: HomeComponent,
  } ];
```

Il comando precedente utilizzato senza parametri,

```
ng generate module home
```

crea semplicemente un modulo all'interno del quale si possono poi aggiungere manualmente uno o più componenti.

L'opzione `--module app.module` è obbligatoria in presenza dell'opzione `--route`

L'opzione `--route home` fa sì che venga creato l'intero componente anche e venga anche aggiornato il file di routing generale `app-routing.module` con l'aggiunta della seguente route:

```
{ path: 'home',
  loadChildren: () => import('./home/home.module')
    .then(mod => mod.HomeModule)
}
```

Cioè, in corrispondenza della richiesta `/home`, anziché specificare il Component da caricare, come specificato nelle routes iniziali, si utilizza il metodo `loadChildren()` che consente di caricare il componente soltanto nel momento in cui l'utente invocherà il path `/home`.

Il metodo `loadChildren()` si aspetta come parametro una function priva di parametri all'interno della quale viene invocata la funzione typescript `import` che restituisce una Promise. In corrispondenza del termine del caricamento viene generato l'evento `then()` la cui funzione di callback deve restituire il **modulo** da caricare. All'interno del modulo c'è il nome del componente.

Senza aver scritto nessuna riga di codice, se si apre l'applicazione e si digita nella barra di navigazione `/home` il router provvederà automaticamente a caricare all'interno di `<router-outlet>` la pagina html del componente home.

Pubblicazione di un progetto Angular su Firebase

Firebase è un servizio di Google Cloud che consente di ospitare diverse tipologie di applicazioni web. Oltre alla pubblicazione statica, consente l'utilizzo di Facility aggiuntive, Gestione Utenti, Gestione Dati su un dB noSql molto simile a mongo.

Prima di pubblicare su Firebase verificare all'interno del file `angular.json` l'impostazione del campo `outputPath` che indica la sottocartella all'interno della quale verranno creati i files di produzione. Questa sottocartella di solito è impostata a `public` oppure `dist` e può essere impostata/modificata manualmente all'interno del file.

Eseguire quindi un build di produzione dell'intero progetto:

```
ng build --configuration production
```

il quale crea la cartella di output definita sopra con all'interno tutti i files necessari per la pubblicazione.

L'opzione `--configuration production` serve a settare a true il flag "production configuration" che ripulisce la app da tutte le informazioni di debug e la rende pubblicabile su un server. Il typeScript viene tradotto in javascript, tutto il routing viene sostituito con dei tag `<a href=".....">`

Creazione del progetto su Firebase

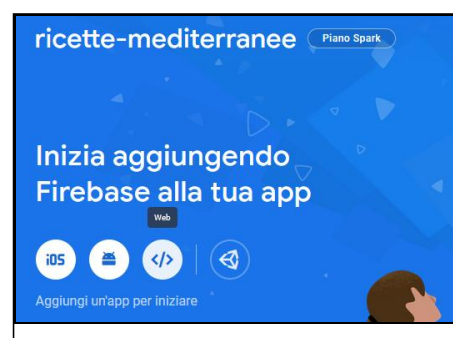
<https://firebase.google.com/>

Per accedere occorre disporre di un **Account Google**

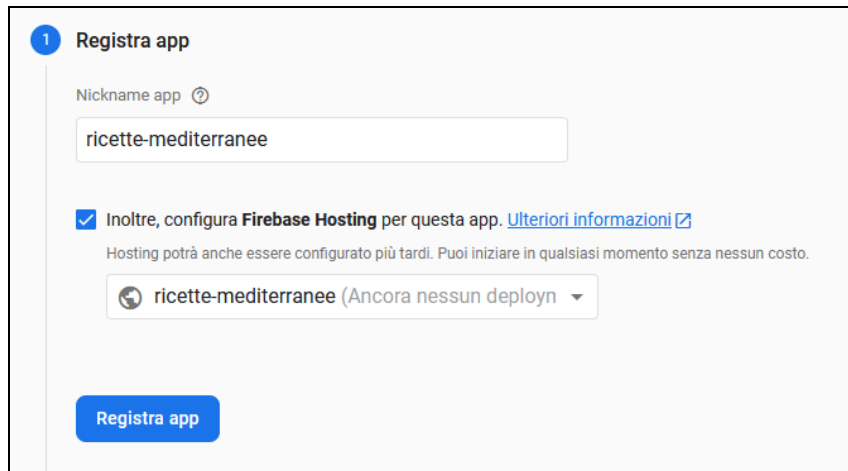
- In alto a destra **Go to Console**
- **Crea un progetto** ed assegnare un nome (anche non univoco, ad esempio `myrecipes`)
- Automaticamente firebase crea un corrispondente **SITE_ID** univoco concatenato un **identificatore numerico** tale da renderlo univoco all'interno del dominio `.web.app`. Questo **SITE_ID** può essere modificato in fase di creazione (ad esempio `robertomana-recipes`) sempre in modo da essere **univoco**, ma NON potrà più essere modificato in seguito.
- Per il nome del progetto ed il **SITE_ID** occorre utilizzare la tipica notazione URL, cioè sono consentiti solo caratteri minuscoli, numeri e trattino per una lunghezza max di 30 caratteri.
- I progetti sono dei semplici contenitori per le APP. In uno stesso progetto possono essere aggiunte più APP
- Eventualmente (**ma non è il caso**) abilitare il servizio di **Google Analytics** che abilita un insieme di funzionalità di test e reporting relativi al progetto. In caso di abilitazione di Google Analytics occorre configurare la località del progetto (Italia).
- **CREA PROGETTO**

Dopo aver creato il progetto selezionare il tipo di APP che si intende realizzare, scegliendo tra:

- IOS
- Android
- **Web App**
- Unity
- Flutter



Si presenta quindi la seguente finestra di richiesta di un **nickname**, che è semplicemente un nome per identificare la app all'interno della firebase console (può essere lo stesso nome del progetto o altro)



- Se nella finestra precedente si seleziona il checkbox ☒ **Firebase Hosting**, vengono automaticamente proposte le varie fasi che devono essere eseguite da linea di comando per la pubblicazione del progetto.
- Se invece NON si seleziona il checkbox, la stessa procedura può essere avviata cliccando sul pulsante di sinistra denominato **Firebase Hosting**.

Script di accesso a Fibase

Per prima cosa viene proposto uno `<script>` contenente le credenziali di accesso a Firebase.

Ci sono due opzioni:

- la **prima opzione** consente invece di creare un file di environment (più sicuro)
- la **seconda opzione** presenta un semplice `<script>` da copia incollare in coda al file `index.html` prima del tag `</body>` in coda al file `index.html` del progetto in modo che quando si esegue la build lo script venga replicato anche nei files di produzione

Se lo scopo è soltanto quello di un hosting statico l'aggiunta di questi script non è necessaria. Salvarsi comunque lo script di environment in locale. Servirà per l'accesso ai dati su firebase

Comandi per la Pubblicazione da linea di comando

Aprire un terminale dalla **cartella principale** di lavoro del progetto Angular (oppure direttamente nella cartella **dist/recipes**, però così facendo nel 2024 in alcuni casi sono poi insorti dei problemi)

- 1) Installare Il Firebase CLI per comunicare a linea di comando con il server

```
npm install -g firebase-tools
```

- 2) **firebase login**

Da fare SOLO in corrispondenza del primo accesso. Viene aperta una finestra del browser in cui scegliere l'account google da utilizzare per l'accesso a Firebase.

Al termine viene visualizzato un messaggio di successo e la finestra del browser può essere chiusa. Sul terminale compare un messaggio che il login è stato effettuato con successo.

In corrispondenza del login firebase invia un token oAuth2 che verrà memorizzato sul pc locale.

Se non ci si collega per molto tempo, il token scade ed in corrispondenza del prossimo accesso con `firebase init` verrà fuori un errore di autenticazione. In tal caso ripetere i seguenti comandi

```
firebase logout
firebase login
// oppure semplicemente
firebase login --reauth
```

3) `firebase init`

Compare il seguente messaggio di cui si chiede conferma

```
You're about to initialize a Firebase project in this directory:
Are you ready to proceed?
```

Viene quindi richiesto quale servizio di firebase intendiamo utilizzare. Tramite frecce e barra spaziatrice selezionare **Hosting: Configure files for Firebase Hosting**

Dal menù successivo selezione **Use an existing project**

E poi selezionare il **SITE_ID** del progetto Firebase creato nei punti precedenti. Questo **SITE_ID** viene memorizzato all'interno del file `.firebaserc` e successivamente non verrà più richiesto.

Viene quindi richiesta la **build output directory** (per default viene proposto `public`)

Digitare **dist/myRecipes** esattamente come indicato nel file `angular.json`

Se invece il terminale è già stato aperto in questa cartella, digitare semplicemente **.**

Se invece è stato aperto in `dist`, digitare `./myrecipes` oppure `myrecipes`

Le ultime tre domande richiedono :

- se l'applicazione è una single page con reidrect automatico di tutte le URL to `index.html` (**Y**)
- se salvare su gitHub tutti i build e deploy (**N**)
- se Firebase deve sovrascrivere il file `index.html` (**N**) Rispondendo Y semplicemente rifà il build

Al termine vengono creati nella cartella di lavoro i files `firebase.json` e `.firebaserc`.

4) `firebase deploy`

La app viene scaricata su firebase e sarà raggiungibile tramite la URL definita dal `SITE_ID` :

`https://robertomana-recipes.web.app/recipes` (con `/recipes` aggiunto automaticamente)

Se su firebase si hanno più progetti attivi, da teminale si possono eseguire i seguenti comandi:

`firebase projects:list` consente di vedere la lista dei progetti attivi

`firebase use robertomana-recipes` consente di 'switchare' sul progetto indicato

Progetti Multisites

Nel momento in cui si dovesse aggiungere al progetto una seconda APP, anche per la seconda APP occorre definire un **SITE_ID** univoco. Per i dettagli si rimanda alla documentazione ufficiale:

`https://firebase.google.com/docs/hosting/multisites`

Pubblicazione del server

Il server dovrà essere pubblicato ad esempio su `onrender.com`. Prima di pubblicare occorre:

- su **angular** nel `data-storage-service` impostare la url dell'onrender server
- sul **crudServer** aggiunegere nella whitelist la seguente entry:

`https://robertomana-recipes.web.app`

Creazione di una apk angular - cordova

La creazione di una apk cordova non interferisce con la creazione di una build per Firebase in quanto ognuno crea i propri file all'interno di cartelle differenti (**dist** per firebase e **www** per cordova)

Dalla cartella di lavoro lanciare il comando **setenv** per l'impostazione delle variabili d'ambiente.

Nel caso di ionic occorre aggiungere la variabile d'ambiente `set _JAVA_OPTIONS=-Xmx512M` che serviva per impostare correttamente le dimensioni della memoria.
Con java 1.8 non serve più anzi sembrerebbe creare problemi.

Step da eseguire

- 1) installare `npm install -g cordova-res`
- 2) Eseguire una build di produzione tramite il comando
`ng build --prod --base-href . --output-path ../www/`

Significato delle opzioni

L'opzione **--prod** serve a settare a true il flag angular di "production configuration" che rende l'apk installabile sul device. Senza questa opzione l'apk non potrà essere installata sul device. Anche con l'opzione **--prod** si tratta però sempre di una apk di debug non adatta per il rilascio su uno store.

L'abbinamento delle opzioni **--prod --release** serve invece per creare una apk non di tipo "debug" ma di tipo "release" rilasciabile sugli stores. Attenzione però che una apk di tipo "release" deve essere controfirmata con le apposite chiavi, altrimenti non solo non sarà pubblicabile sugli stores, ma non sarà nemmeno installabile sui propri device.

L'opzione **--base-href .** should be used to set the base reference to "." as absolute paths are not well handled by Cordova

L'opzione **--output-path** indica il percorso dove salvare la build di produzione. La documentazione raccomanda di salvare la build di produzione in una cartella diversa rispetto alla cartella di lavoro di angular in modo da non creare interferenza fra le librerie di angular e quelle di cordova.

- 3) Creare una nuova applicazione cordova e sostituire la cartella **www** con la cartella appena creata al punto precedente.
- 4) Aggiungere in coda ad index.html (prima del </body>) il link a cordova.js.

```
<script type="text/javascript" src="cordova.js"></script>
```

- 5) Lanciare quindi i soliti comandi cordova:

```
cordova platform add android
cordova build
```

L'apk viene salvata nel percorso `\platforms\android\app\build\outputs\apk\debug`

Esecuzione diretta della app

Come per tutte le app cordova è possibile eseguirla direttamente senza doverla installare su un device. A tal fine occorre :

- Installare `npm install -g native-run`
- Aprire un emulatore oppure collegare un device
- Lanciare uno dei seguenti comandi:

```
cordova run android // build + run su emulatore o device
cordova run android --no-build // solo run (installa soltanto)
chrome://inspect
```